

МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение
высшего образования
«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
АЭРОКОСМИЧЕСКОГО ПРИБОРОСТРОЕНИЯ»

Кафедра №43

А.А. Попов

Методические рекомендации к выполнению курсового проекта
по курсу
“Программирование встроенных приложений”

Содержание:

1	ОПИСАНИЕ КУРСОВОГО ПРОЕКТА.....	2
1.1	ЭТАПЫ КУРСОВОГО ПРОЕКТА	2
1.2	СОДЕРЖАНИЕ ПОЯСНИТЕЛЬНОЙ ЗАПИСКИ	3
2	ЗАДАНИЕ НА КУРСОВОЕ ПРОЕКТИРОВАНИЕ.....	4
3	ЭТАПЫ РАЗРАБОТКИ ВСТРОЕННОГО ПРИЛОЖЕНИЯ.....	13
4	ПОРЯДОК РАБОТЫ С PROTEUS 8.....	20
4.1	ВЗАИМОДЕЙСТВИЕ МОДЕЛИ В PROTEUS СО СРЕДОЙ РАЗРАБОТКИ PROCESSING ПОСРЕДСТВОМ ИНТЕРФЕЙСА UART	24
4.2	ПОРЯДОК ПОДКЛЮЧЕНИЯ LCD ЭКРАНА В PROTEUS НА ПРИМЕРЕ ЧАСОВ RTC	32
5	ПОРЯДОК РАБОТЫ С СИМУЛЯТОРОМ KEIL	36
6	ЛИТЕРАТУРА	41

1 Описание курсового проекта

Цель курсового проекта: получение практических навыков программирования встраиваемых приложений в ходе разработки конкретного прототипа устройства.

Курсовой проект заключается в самостоятельном проектировании прототипа прибора на основе изученных подсистем МК. Поскольку курсовое проектирование выполняется удалённо, без отладочной платы, то темы заданий подразумевают выполнение проекта на симуляторе в среде Keil uVision5, САПР Proteus и среде разработки Processing.

Подключение различных устройств: светодиодов, кнопок, экрана, различных датчиков, интерфейсных модулей к микроконтроллеру STM32F103C8T6 осуществляется в среде Proteus. На оценку отлично обязательно использование иерархии: интерфейс управления в среде Processing >> управление посредством команд через интерфейс UART прототипом прибора собранного в Proteus >> исполняемый код микроконтроллера STM32F103C8T6, собранный в Keil. На оценку хорошо достаточно модели в Proteus.

На оценку удовлетворительно достаточно симулятора среды Keil. Для имитации нажатия кнопок или изменения сигнала на линиях в/в используйте скрипт отладчика Keil. Таким образом виртуальные кнопки – меняют уровень сигнала на входе портов ввода/вывода (ПВВ); светодиоды – эмуляторы сигналов на выходах ПВВ, которые наблюдаем в Logic Analyzer; экран с автоматической прокруткой для отображения текста – Debug (printf) Viewer.

1.1 Этапы курсового проекта

На выполнение одной темы предусматривается совместная работа нескольких студентов. Контроль выполнения курсового проекта разбит на следующие этапы:

1) Разработка технического задания (ТЗ) на прототип прибора с указанием: условий применения; функциональных требований; требований к формату обмена с внешней средой (пользователем, компьютером, памятью, экраном и т.п.); привести пример структуры данных для обмена (формат протокола). Требования представить в виде таблицы. Разработать функциональную модель системы. Функциональная модель содержит четыре компонента:

1. Операции – преобразование данных.
2. Участники операций – производители и потребители значений.
3. Хранилища данных.
4. Потоки данных – отношения между операциями, участниками и хранилищами.

- 2) Разработка структурных схем прибора как программно-аппаратной системы, которая включает в себя:
- а) структурную схему соединения модулей прототипа прибора с указанием используемых портов в/в МК, подключаемых к ним периферийных устройств, сигналов интерфейсов связи, используемых протоколов связи, напряжений питания;
 - б) граф-схемы взаимодействия элементов системы: диаграммы состояний подсистем и условий перехода между ними; диаграммы последовательности взаимодействия подсистем; блок-схемы алгоритмов обработки данных.
- Разработка и отладка программного обеспечения МК.
- 3) Разработка контрольных тестов прототипа для проверки соответствия техническому заданию. Осуществить тестирование и отладку.
- 4) Оформление пояснительной записки.
- 5) Защита работы – общий доклад всей группы студентов по результатам разработки с обоснованием принятых решений длительностью до 5 мин. Архив проекта (zip) скопировать на компьютер преподавателя, продемонстрировать компиляцию и корректное прохождение прототипом разработанных тестов.

1.2 Содержание пояснительной записки

Объём ПЗ от 10 до 40 стр. ПЗ должна содержать:

1. Задание на курсовое проектирование. Назначение и режимы работы разрабатываемого прототипа.
2. Техническое задание на прототип. Представить в таблице список требований к системе по разделам: требования к реализации (какой язык программирования и какие библиотеки будем использовать); функциональные требования; аппаратные требования. Описать форматы проектируемых команд, форматы отображения информации, формат обмена пакетами данных (транспортный уровень), форматы сигналов, форматы кнопок и т.п. Представить диаграмму вариантов использования; список функций, возлагаемых на прототип прибора.
3. Проектирование архитектуры системы. Привести и описать контекстную диаграмму системы. Привести структурную схему соединений модулей устройства, с указанием номеров линий связи, портов, названий периферийных устройств. Указать диапазоны напряжения сигналов, форматы протоколов обмена на аппаратном уровне и их скорости. Привести схему архитектуры системы, описать организацию взаимодействия элементов в системе.

4. Разработка программного обеспечения для элементов архитектуры системы. Используя граф-схемы описать алгоритмы инициализации устройств, алгоритмы работы. Описать диаграмму состояний системы и условия перехода между ними. Описать диаграммы последовательности взаимодействия в системе. Описать программные интерфейсы разрабатываемых программных модулей.
5. Разработка контрольных тестов и тестирование устройства на соответствие ТЗ. В виде таблицы привести последовательность действий и образец реакций на каждое действие. Обосновать полноту разработанных тестов.
6. Произвести экономическую оценку производства прототипа используя цены с сайта aliexpress.ru.
7. Привести в приложении код основных программных модулей.

Пояснительная записка в одном файле pdf представляются в личном кабинете студента до конца сессии.

2 Задание на курсовое проектирование.

Темы курсового проектирования показаны в таблице 1. Одна тема рассчитана на совместную работу двух человек. Порядок распределения тем определить старосте группы и выслать преподавателю список группы с закрепленными темами. В примерах в Processing, где используется шрифт, удаляйте расширение шрифта, часть `.ttf`. Допускаются собственные темы на собственной отладочной плате по согласованию с преподавателем.

Таблица 1 – Перечень тем курсовой работы

№ п/ п	Темы
Темы на отлично, выполняются в Processing и Proteus.	
1	Выносной пульт управления часами с будильником. В качестве ПО часов с будильником на ПК выступает среда Processing, пример https://processing.org/examples/clock.html которой по интерфейсу UART управляет выносной пульт управления – микроконтроллер в среде Proteus. Выносной пульт на основе интерфейса UART, LCD и кнопок управляет состоянием часов на ПК. Реализовать командную строку на LCD панели. После ввода команды, нажатием кнопки, отправить код команды по интерфейсу UART на скорости 115200 бод на ПК (команду управления часами) и перевести каретку на новую строку на экране (сохраняется история команд). При введении команды, не соответствующей формату, выводить ошибку на LCD, на ПК ничего не отправлять. Форматы команд определить самостоятельно. Разработанный набор команд должен: устанавливать часы в заданное время, устанавливать дату, устанавливать будильник, запускать и останавливать часы/будильник, считывать текущее время часов, считывать время будильника. В каждой команде должна быть как минимум одна опция. Не указанные ограничения и форматы доопределить самостоятельно.
2	Выносной пульт менеджера папок и файловой системы на сервере. В качестве сервера выступает ПК к которому по интерфейсу UART подключён пульт. Используя пример https://processing.org/examples/loadfile2.html, в среде Processing написать ПО которое по полученным командам от выносного пульта будет считывать файловую систему ПК и отправлять считанный список файлов на пульт контроля. На основе интерфейса UART, LCD и кнопок реализовать выносной пульт контроля состояния файлов и папок на ПК. По интерфейсу UART, на скорости 9600 бод, принять от ПК список файлов текущей просматриваемой папки, сформировать и отобразить список папок и файлов на LCD. Пользователь с помощью кнопок выбирает папку на экране LCD и её код передаётся на ПК, который возвращает содержимое выбранной папки. Разработать формат отображения и соответствующие им коды приёма/передачи. Не указанные форматы доопределить самостоятельно.
3	Пульт открытия дверей. В качестве удаленного сервера управления дверями выступает ПК к которому по интерфейсу UART подключён пульт. Используя пример: https://processing.org/examples/charactersstrings.html, в среде Processing написать эмулятор управления дверями на режимном объекте. Пульт-микроконтроллер по интерфейсу UART передаёт/принимает от сервера безопасности зашифрованные коды. Пульт принимает ключ доступа и передаёт команды: открытия, закрытия, блокировки, разблокировки, текущего состояния дверей. На основе ключа реализовать шифрование канала связи. В случае неверного ключа переводить двери в режим блокировки. Форматы введения ключа и команд определить самостоятельно. Пример работы с последовательным портом на ПК https://processing.org/reference/libraries/serial/Serial_bufferUntil_.html.

4

Система связи через транслятор в код Морзе.

В качестве удаленного сервера выступает ПК к которому по интерфейсу UART подключён транслятор в код Морзе. Используя пример: <https://processing.org/examples/charactersstrings.html>, в среде Processing написать приложение со строкой ввода символов, ручкой регулировки скорости передачи кода Морзе. Пользователь-учитель на ПК устанавливает скорость передачи и вводит текст которые передаются по интерфейсу UART в прибор.

Прототип принимает по интерфейсу UART текст и скорость, воспроизводит текст в коде Морзе через динамик (SPEAKER ) и светодиод с заданной скоростью.

Пользователь-ученик, прослушав звуки выбирает вариант переданного текста из 4х предложенных. В случае ввода неверного варианта на экране показать правильный, передать результат для учёта на сервер. Не указанные ограничения и форматы доопределить самостоятельно.

5	Удалённые вычисления на калькуляторе выражений. Реализовать в Proteus удалённого клиента инженерного калькулятора выражений, который производит вычисления на сервере в Processing. Ввод выражения для вычисления осуществляется, используя кнопки и LCD, далее выражение по интерфейсу UART передаётся на сервер-ПК, где вычисляется и сохраняется в журнале вычислений, а результат передаётся обратно в калькулятор и отображается на экране. Вводимые символы: '0', '1', '2', '3', '4', '5', '6', '7', '8', '9', '.', '=', '+', '-', '*', '/', '(', ')', 'ln'. Журнал вычислений должен быть доступен для просмотра в Processing, и при выборе выражения передавать результат его вычисления клиенту для просмотра. Не указанные ограничения и форматы доопределить самостоятельно.
6	Приставка-генератор тонального сигнала – DTMF. Реализовать в Proteus на микроконтроллере генерацию ШИМ тонального сигнала, который воспроизводится через динамик (SPEAKER ). Используя кнопки и LCD реализовать ввод символа и воспроизведение его тонального кода. Подобный интерфейс реализовать в Processing, где по нажатию кнопки воспроизвести соответствующее ей звучание тонального сигнала сначала в Processing как в примере https://processing.org/reference/libraries/sound/AllPass_process_.html. А затем передать код в приставку по интерфейсу UART и воспроизвести там. Частота дискретизации ШИМ вводится как отдельная настройка. Формат отображения значения частоты дискретизации ШИМ, символа, а также кнопки, ограничения по настройкам определить самостоятельно.
7	Игра «крестики-нолики» (2). Один игрок играет на прототипе прибора в Proteus, второй игрок в приложении Processing. В прототипе и в приложении на экране в поле 3x3 реализовать игру. Пока одна сторона не сделала ход, другая сторона ожидает заданный интервал времени, по прошествии которого ход делается принудительно случайным образом. Вести общий счёт по играм. Управление продумать самостоятельно. Не указанные ограничения и форматы доопределить самостоятельно. Тип экрана для прототипа выбрать самостоятельно.
8	Игра «морской бой» (2). Один игрок играет на прототипе прибора в Proteus, второй игрок в приложении Processing. В прототипе и в приложении на экране в поле 4x4 реализовать игру с двумя кораблями размером в одну клетку. Управление продумать самостоятельно, тип экрана для прототипа выбрать самостоятельно. Не указанные ограничения и форматы доопределить самостоятельно. Добавить звуковое сопровождение.

9

Удалённо управляемая гирлянда из 24х светодиодов - «бегущий огонёк».

В приложении Processing разработать пульт управления светодиодами на исполнительном устройстве в Proteus к которому подключены светодиоды. Реализовать режимы движения «огонька» с заданной скоростью согласно таблице. Решить самостоятельно, какие кнопки создать, и с какими функциями.

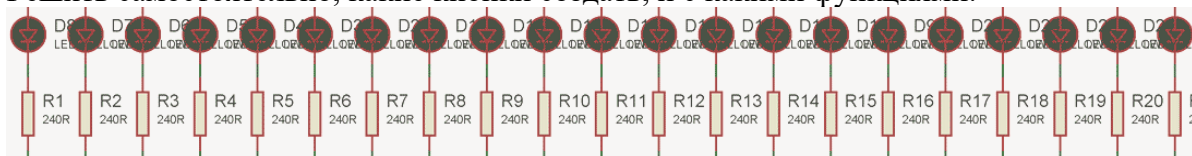
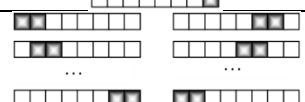


Таблица режимов:

1. «Бегущий огонёк» светодиода слева направо и обратно



2. «Бегущий огонёк» из 2х светодиодов слева направо и обратно



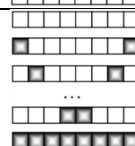
3. 3-х светодиодов слева направо в одну сторону



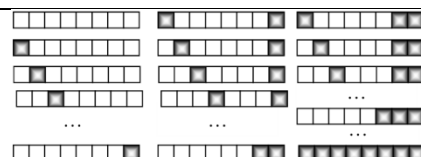
4. «Разбегающиеся огоньки»



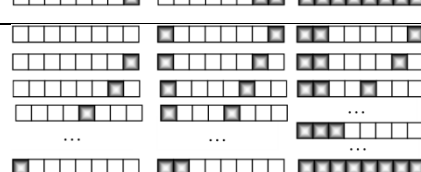
5. «Сталкивающиеся огоньки»



6. «Накопление» слева направо



7. «Накопление» справа налево



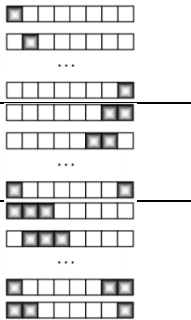
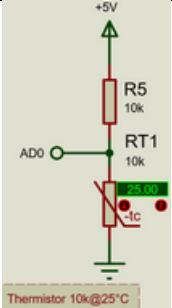
8. Последовательное включение и выключение всех светодиодов к центру



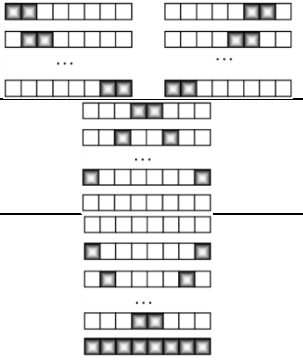
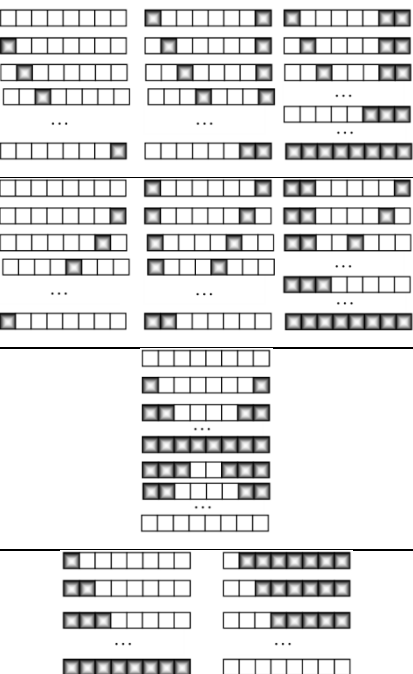
9. Последовательное зажигание и выключение светодиодов

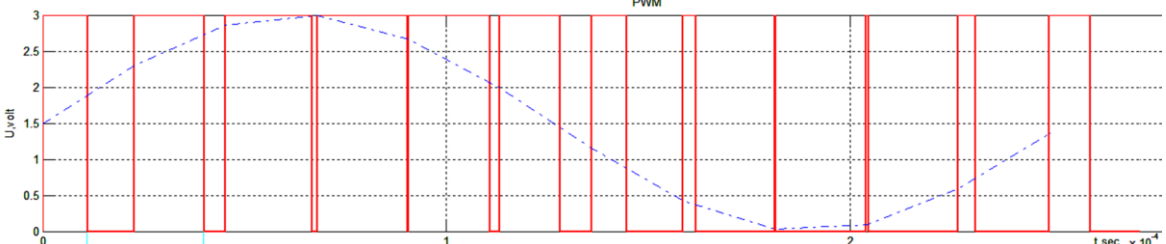


Темы на хорошо, выполняются в Proteus	
1	Транслятор в код Морзе. Посредством Virtual Terminal через UART реализовать ввод символов. Далее по кнопке начать передачу, передать сообщение в коде Морзе используя светодиод и динамик (SPEAKER ). Скорость передачи 120 знаков в минуту. Решить самостоятельно, какие кнопки создать, и с какими функциями. Не указанные ограничения и форматы доопределить самостоятельно.
2	Часы с будильником. Реализовать часы в формате ЧЧ:ММ:СЕК-/день недели/-/число/. Установку времени часов и будильника осуществлять через кнопки. Звук воспроизводить через динамик (SPEAKER ). Решить самостоятельно, какие кнопки создать, и с какими функциями.
3	Калькулятор. Реализовать калькулятор. Вводимые символы: '0', '1', '2', '3', '4', '5', '6', '7', '8', '9', '.', '=', '+', '-', '*', '/'. Реализовать временную память MS, MR. Обязательно реализовать контроль корректности ввода. Не указанные ограничения и форматы доопределить самостоятельно.
4	Генератор ШИМ заданной звуковой частоты. На основе ШИМ реализовать генерацию синусоидального сигнала. Настройку частоты следования ШИМ импульсов, частоты модулируемой синусоиды осуществлять с помощью кнопок и LCD панели. Формат отображения значения частоты ШИМ, частоты синусоиды, а также кнопки, ограничения по настройкам определить самостоятельно. Звук ШИМ синусоиды воспроизводить через динамик (SPEAKER ).
5	Таймер обратного отсчёта. На основе кнопок и LCD реализовать таймер обратного отсчёта времени (от 2 сек. до 10 мин), с выводом каждой десятой секунды обратного отсчета. Во время работы мигать светодиодом с частотой 2 Гц. По истечению времени перестать мигать и включать зуммер (SPEAKER ). Какие кнопки использовать, и какими функциями наделить решить самостоятельно.
6	Логический двоичный калькулятор. Используя индикатор на четыре цифры 7SEG-MPX4-CA, реализовать простейший 4х разрядный логический калькулятор чисел в двоичной системе счисления. Ввод знаков 0, 1, & (и), ~ (не), (или), ^ (мод2), = осуществлять через кнопки. Ввод знаков сопровождать зажиганием светодиода на полсекунды. Решить самостоятельно, какие кнопки создать, и с какими функциями.

7	Гирлянда из 30-ти светодиодов - «бегущий огонёк». Подключить адресную светодиодную ленту APA102CX30 посредством интерфейса SPI к микроконтроллеру. Реализовать движение «огонька» влево или вправо с заданной скоростью (переключение линий поочередно) как показано в примере ниже, двух, трёх, четырёх огней по выбору. Цвет, скорость, режим движения (влево, вправо) и количество огоньков (1, 2, 3, 4), задавать кнопками. Текущие значения настроек выводить на LCD. Решить самостоятельно, какие кнопки создать, и с какими функциями. «Бегущий огонек» для одного светодиода слева направо 2 светодиода справа налево 3-х светодиодов слева направо 
8	Датчик температуры - термистор. Подключить термистор NTC посредством ADC. Используя АЦП микроконтроллера считать значение напряжения на выходе AD0, пересчитать его в температуру и вывести на экран LCD. Решить самостоятельно, какие кнопки создать, и с какими функциями. 
9	Датчик давления. Подключить датчик давления BMP280 посредством интерфейса I²C. Считать с датчика значения давления и температуры, пересчитать в высоту согласно ГОСТ 4401-81 и вывести на экран LCD. Решить самостоятельно, какие кнопки создать, и с какими функциями.
10	Датчик температуры. Подключить датчик температуры DS18B20 посредством интерфейса 1-wire. Используя интерфейс UART микроконтроллера считать значение температуры с датчика и вывести на экран LCD. Решить самостоятельно, какие кнопки создать, и с какими функциями.
Темы на удовлетворительно, выполняются в Keil	
1	Таймер обратного отсчёта. Используя окно отладчика (Debug (printf) Viewer) реализовать таймер отсчёта времени (от 5 сек. до 2 мин), с выводом каждую секунду в окно отладчика обратный отсчет секунд. Во время работы таймера мигать светодиодом с частотой 5Гц (переключать линию PC13), во время простоя с частотой 1Гц. В момент окончания отсчёта таймера зажечь светодиод на 3 сек. Ввод времени для отсчёта и запуск таймера осуществлять через кнопки, созданные скриптом отладчика. Реализовать режимы однократного и циклического запуска таймера (интервальный режим). Решить самостоятельно, какие кнопки создать, и с какими функциями.

2	Часы. Реализовать вывод в окно отладчика (Debug (printf) Viewer) часов в формате ЧЧ:ММ:СЕК-/-день недели/-/число/. Вывод осуществлять каждые две секунды. Во время работы часов мигать светодиодом с частотой один раз в две секунды (переключать линию PC13) синхронно с выводом в окно отладчика. Установку времени осуществлять через кнопки, созданные скриптом отладчика. При нажатии кнопок установки времени мигать светодиодом с частотой 5Гц в течение 2х секунд. Решить самостоятельно, какие кнопки создать, и с какими функциями.
3	Логический двоичный калькулятор. Используя окно отладчика (Debug (printf) Viewer) как экран, реализовать простейший логический калькулятор чисел в двоичной системе счисления. Ввод знаков 0, 1, & (и), ~ (не), (или), ^ (мод2), = осуществлять через кнопки созданные скриптом отладчика. Ввод знаков сопровождать зажиганием светодиода на полсекунды (переключать линию PC13). Решить самостоятельно, какие кнопки создать, и с какими функциями.
4	Двоичный калькулятор. Используя окно отладчика (Debug (printf) Viewer) как экран, реализовать простейший калькулятор чисел в двоичной системе счисления. Ввод знаков 0, 1, +, -, *, /, = осуществлять через кнопки, созданные скриптом отладчика. Ввод знаков сопровождать зажиганием светодиода на полсекунды (переключать линию PC13). Решить самостоятельно, какие кнопки создать, и с какими функциями.
5	Игра «Салки». Считать, что к десяти линиям ПВВ В подключена гирлянда из 10 светодиодов, причём высокий уровень на линии зажигает светодиод, низкий гасит. Реализовать случайное или управляемое (закон управления придумать самостоятельно) движение по гирлянде «огонька»-«ловец» с заданной скоростью (скорость задавать с помощью кнопок). И движение «огонька»-«игрок» который управляется пользователем кнопками влево и вправо. Задача ловца догнать и поймать игрока. Решить самостоятельно, какие кнопки создать, и с какими функциями. Стратегию поведения «ловца» определить самостоятельно.
6	Гирлянда из 8-ми светодиодов - «бегущий огонёк». Считать, что к линиям ПВВ А подключены светодиоды, причём высокий уровень на линии зажигает светодиод, низкий гасит. Реализовать движение «огонька» влево или вправо с заданной скоростью (переключение линий поочередно) как показано в примере ниже, двух, трёх, четырёх огней по выбору. Скорость, режим движения (влево, вправо) и количество огоньков (1, 2, 3, 4), задавать через PORTB и кнопки, созданные скриптом отладчика. Текущие значения настроек выводить в окно отладчика (Debug (printf) Viewer). Решить самостоятельно, какие кнопки создать, и с какими функциями. Примеры возможных режимов: «Бегущий огонек» для одного светодиода слева направо 2 светодиода справа налево 3-х светодиодов слева направо

7	Гирлянда из 8-ми светодиодов - «разбегающиеся огоньки». Считать, что к линиям ПВВ В подключены светодиоды, причём высокий уровень на линии зажигает светодиод, низкий гасит. Реализовать движение «огоньков» в одном из трёх режимов с заданной скоростью. Один или два «огонька» по выбору. Скорость, режим движения (с отражением влево-вправо, разбегающийся, сталкивающийся) и количество огоньков (1, 2), задавать через PORTA и кнопки, созданные скриптом отладчика. Текущие значения настроек выводить в окно отладчика (Debug (printf) Viewer). Решить самостоятельно, какие кнопки создать, и с какими функциями. Режимы работы: «Бегущий огонек» из 2х светодиодов слева направо и обратно «Разбегающиеся огоньки» «Сталкивающиеся огоньки» 
8	Гирлянда из 8-ми светодиодов - «накопление огоньков». Считать, что к линиям ПВВ А подключены светодиоды, причём высокий уровень на линии зажигает светодиод, низкий гасит. Реализовать накопление «огоньков» в одном из четырёх режимов с заданной скоростью. Скорость, режим работы (указаны ниже) и количество одновременно скользящих огоньков (1 или 2), задавать через PORTB и кнопки, созданные скриптом отладчика. Текущие значения настроек выводить в окно отладчика (Debug (printf) Viewer). Решить самостоятельно, какие кнопки создать, и с какими функциями. Режимы работы: «Накопление» слева направо «Накопление» справа налево Последовательное включение и выключение всех светодиодов к центру Последовательное зажигание и выключение светодиодов 

9	Транслятор числа в код Морзе. Используя 5 линий порта в/в реализовать ввод шестнадцатеричных чисел в двоичной системе (посредством кнопок, созданных скриптом отладчика), где линии с 1 по 4 устанавливает отдельный разряд, а пятая линия ввод установленного числа. Вывод четырёхразрядного числа во время установки в окно отладчика (Debug (printf) Viewer) осуществлять в формате bxxxx – hX (пример b1101 – hD). Далее по кнопке ввод начать передачу введенного шестнадцатеричного числа в коде Морзе по линии PC13. Скорость передачи 120 знаков в минуту. Решить самостоятельно, какие кнопки создать, и с какими функциями.
10	Приёмник десятичного числа в коде Морзе. Используя одну линию в/в для приёма сигнала кода Морзе, осуществить распознавание принятого десятичного числа. Скорость приёма 400 знаков в минуту. Числа могут быть многозначными. Печать принятого числа осуществлять в окно отладчика (Debug (printf) Viewer). Используя скрипт отладчика задать 10 кнопок с функцией генерации на линии приёма кодов чисел от 0 до 9.
11	Генератор прямоугольных импульсов. Реализовать генерацию прямоугольных импульсов на выходах PC13 и PC14 (инвертированный) заданной частоты и скважности. Частоту и скважность каждого выхода задавать кнопками, созданными скриптом отладчика, значения выводить в окно отладчика (Debug (printf) Viewer). Решить самостоятельно, какие кнопки создать, и с какими функциями.
12	Генератор ШИМ синусоидального сигнала. Реализовать генерацию ШИМ синусоидального сигнала на выходе PC13. Частоту дискретизации ШИМ и частоту синусоиды задавать кнопками, созданными скриптом отладчика, значения выводить в окно отладчика (Debug (printf) Viewer). Решить самостоятельно, какие кнопки создать, и с какими функциями. 

3 Этапы разработки встроенного приложения

Основные этапы разработки (см. рис. 1) рассмотрим на примере создания часов с будильником.

В рамках разрабатываемой модели часов с будильником выделяются следующие исполнители (акторы): пользователь; дисплей; динамик для воспроизведения звука; микроконтроллер как управляющее устройство (см. рис. 2).

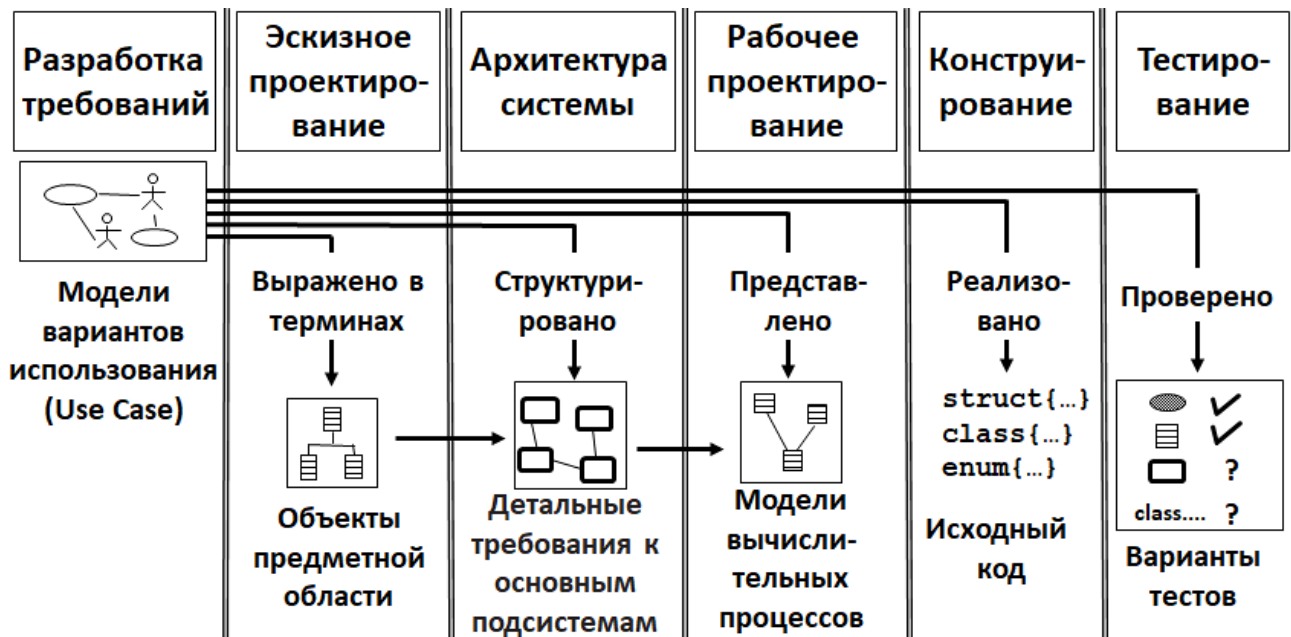


Рисунок 1 – Этапы разработки встроенного приложения

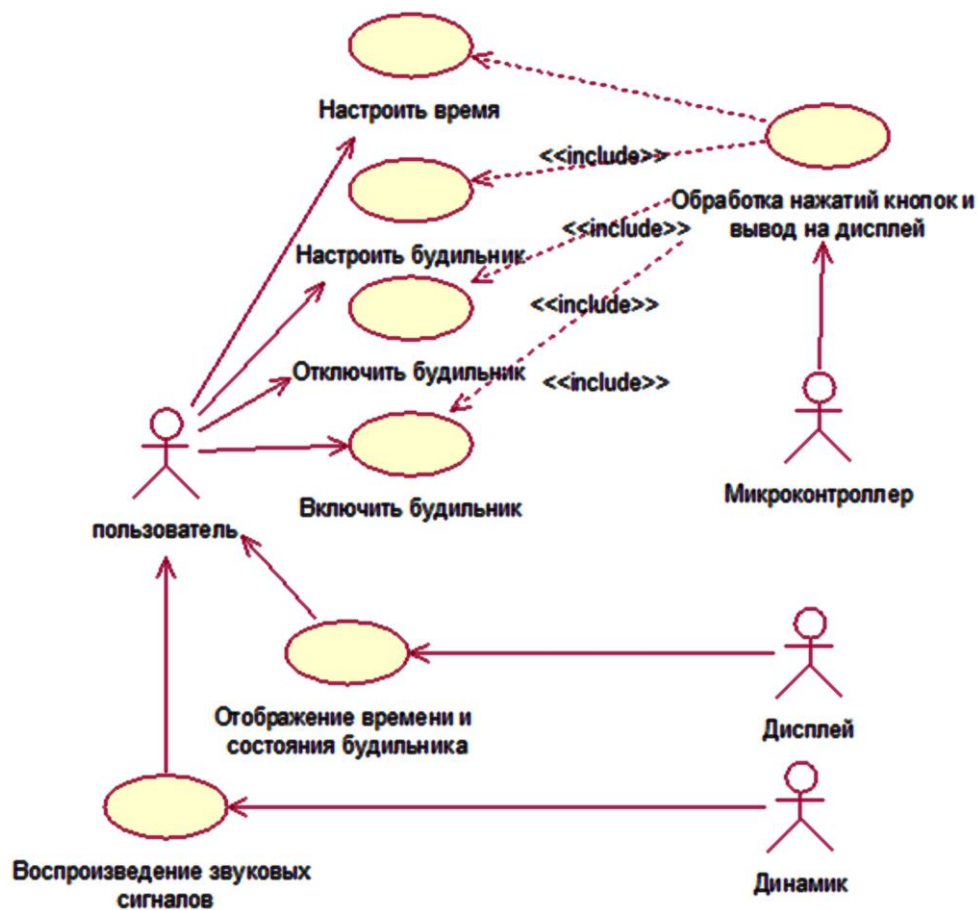
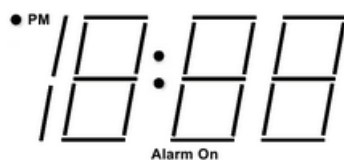


Рисунок 2 – Диаграмма вариантов использования часов

Основные функции часов с будильником:

- Время отображается в виде четырех цифр в 12-часовом формате – ЧЧ:ММ, используется индикатор, чтобы различать AM и PM;



- Используется несколько кнопок для установки времени на часах и будильнике;
- Включаем и выключаем будильник с помощью кнопок;
- Когда будильник активирован, индикатор готовности (Alarm On) горит;
- Отдельный динамик обеспечивает звуковую сигнализацию.

В таблице 2, определены допустимые значения для индикатора времени. В таблице 3 определены кнопки для управления часами.

Таблица 2 – Допустимые значения индикатора времени

Индикатор времени	Допустимый диапазон
Минуты	[0:59]
Часы	[0:12]
Индикатор интервала	[AM, PM]
Состояние будильника	[Alarm On, Alarm Off]

Таблица 3 – Назначения кнопок

Название кнопки	Назначение
Change time	Включить режим изменения времени
Set alarm	Включить режим установки времени будильника
Minute+	Добавить одну минуту, вывести результат
Hour+	Добавить один час, вывести результат
Alarm On	Включить будильник
Alarm Off	Отключить будильник

Сведём все требования к часам в одну таблицу 4.

Таблица 4 – Требования к прототипу часов с будильником

Управляющие входы	6 кнопок: установить время, установить будильник, часы+, минуты+, будильник включен, будильник выключен.
Выходы	Звуковой пьезоэлемент, сигнализирующий срабатывание будильника. Экран для индикации 6-ти цифр времени в 12-часовом формате ЧЧ:ММ. Индикатор AM/PM. Индикатор вкл/выкл будильника.

Функции режимов	1) Режим по умолчанию: на дисплее отображается текущее время, при этом, один раз в секунду, две разделительные точки между часами и минутами мигают. Будильник отключен. 2) В режиме установки/изменения времени, кнопки часов и минут используются для изменения времени часов и будильника соответственно. Нажатие одной из этих кнопок увеличивает час / минуту на единицу. 3) Режим изменения времени: включается нажатием и удержанием кнопки «Change time». Позволяет с помощью кнопок настроить новое время. 4) Режим установки будильника: включается нажатием и удержанием кнопки «Set alarm». Позволяет с помощью кнопок установить время будильника. 5) Режим воспроизведения будильника: включается при равенстве времени часов и будильника. В этом режиме мигает индикатор времени и включен зумер в течении минуты. 6) Кнопка включить будильник «Alarm On»: переводит часы в состояние с включенным будильником, включает индикатор готовности будильника. Когда текущее время достигает времени будильника, включается режим воспроизведения будильника. 7) Кнопка отключить будильник «Alarm Off»: отключение режима воспроизведения будильника и режима «Alarm On», перевод часов в режим по умолчанию.
Особенности	1) При включенном режиме изменения времени нельзя установить будильник до тех пор, пока время не будет изменено. Также, при включенном режиме установки будильника, нельзя изменить время, пока настройка будильника не будет сохранена. 2) При изменении настроек вывод текущего времени приостанавливается. 3) В режиме будильника нельзя установить время равное текущему.
Питание	Питание от сети переменного тока через стандартный блок питания (USB адаптер)
Размеры и вес	170x70*17 мм, с весом менее 300 грамм.

Из проведенного анализа следует, что необходимы: экран для индикации; кнопки; динамик. В качестве экрана выберем LCD LM016L, хотя возможно использовать индикатор 7SEG-MPX4-CA. Общая схема подключения устройств (кнопок, LCD и динамика) показана на рисунке 3.

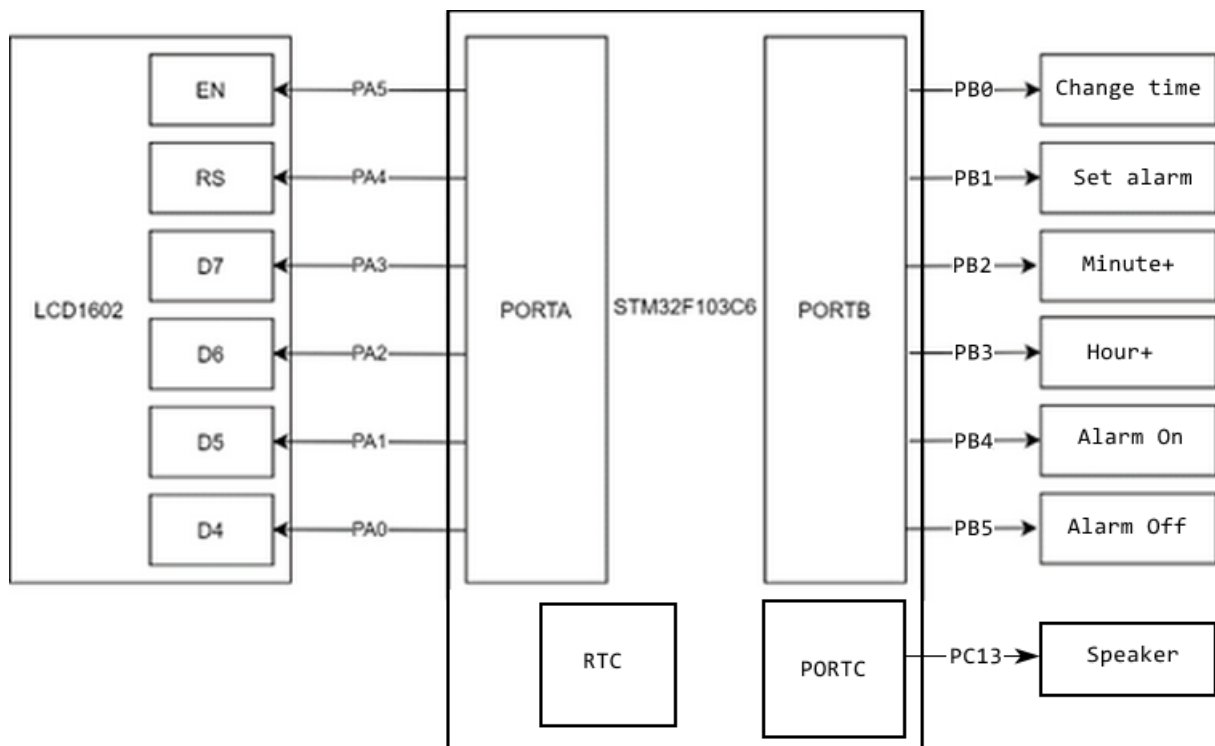


Рисунок 3 – Структурная схема подключения периферийных устройств к МК

Начнём анализ с общей схемы взаимодействия исполнительных механизмов в нашем приложении, причём детализируем понятия, введенные пользователем на этапе постановки задачи (ТЗ). Обобщённая диаграмма последовательности взаимодействия показана на рисунке 4.

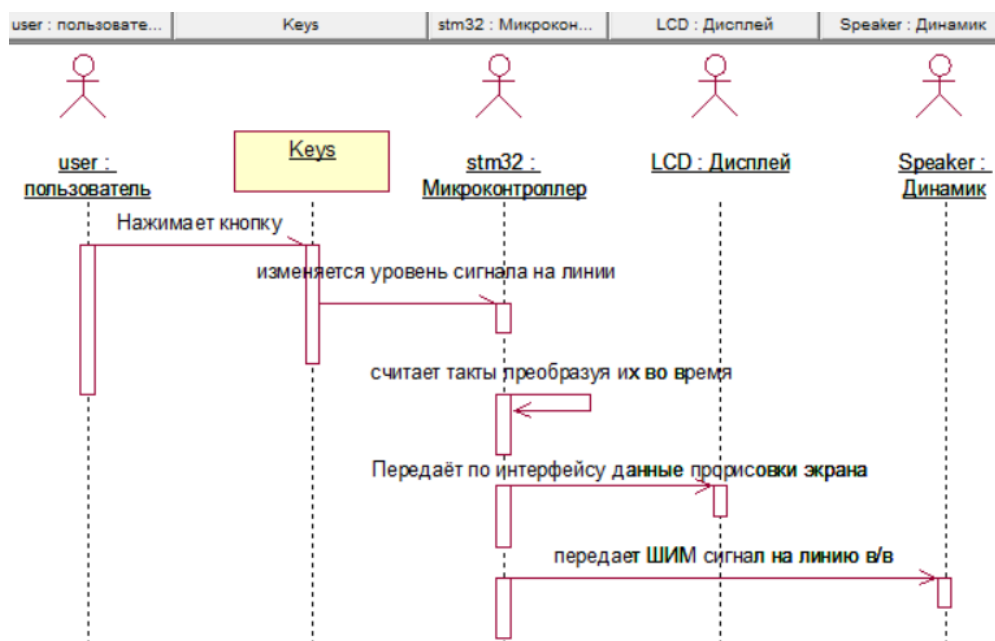


Рисунок 4 – Диаграмма последовательности взаимодействия элементов

Анализ диаграммы (см. рис. 4) показывает, что:

1) Необходим опрос состояния линий кнопок, следовательно определим интервал опроса исходя из функциональных возможностей человека. Высокая скорость ввода 600 знаков в минуту, соответствует вводу одного знака за 100 мсек. Соответственно опрос линий один раз в 50 мсек достаточен для исключения дребезга. Для опроса состояния кнопок с интервалом в 50 мсек используем системный таймер.

2) Необходим точный генератор импульсов для отсчёта времени, используем LSE (32768 Гц) и модуль RTC.

3) Для уменьшения энергопотребления нужно переводить МК в режим сна между интервалами опроса кнопок.

4) Для формирования звука с помощью ШИМ необходима настройка таймера и генерация массива отсчётов воспроизводимого звука.

5) Для работы LCD необходима предварительная настройка интерфейса.

Следующей итерацией детализируем механизм цикла работы часов (см. рис. 5).

Выделим следующие основные управляющие структуры:

- функция `main()` с бесконечным циклом;
- структура `Mechanism`, реализующая общий механизм часов, содержит массивы: счётчиков текущего времени (часы, минуты); времени будильника (часы, минуты); состояние АМ/РМ; будильник вкл./выкл.; функции, реализующие закон изменения массивов счётчиков;
- структура `Display`, содержит функции прорисовки текущего состояния часов два раза в секунду (мигание точек-разделителей часов и минут);
- структура `Keypad` опроса состояния кнопок;
- структура `Buzzer` для воспроизведения звука.

Исходя из вышеизложенного построим диаграмму активности для функции `main` (см. рис. 6).

Теперь получив общее представление о ходе работы программы часов можно начать детальную проработку управляющих структур. Каждую структуру управления можно оформить в виде пары файлов (заголовочного `.h`) и (реализации `.c`) в которых будут определены требуемые для функционирования структуры переменные и функции, а можно всё свалить в один файл вместе с главной функцией `main`.

В данном примере разобрана только основная функция `main()` в которую входят функции структуры `Mechanism`. Далее по аналогии нужно разобраться с работой остальных структур.

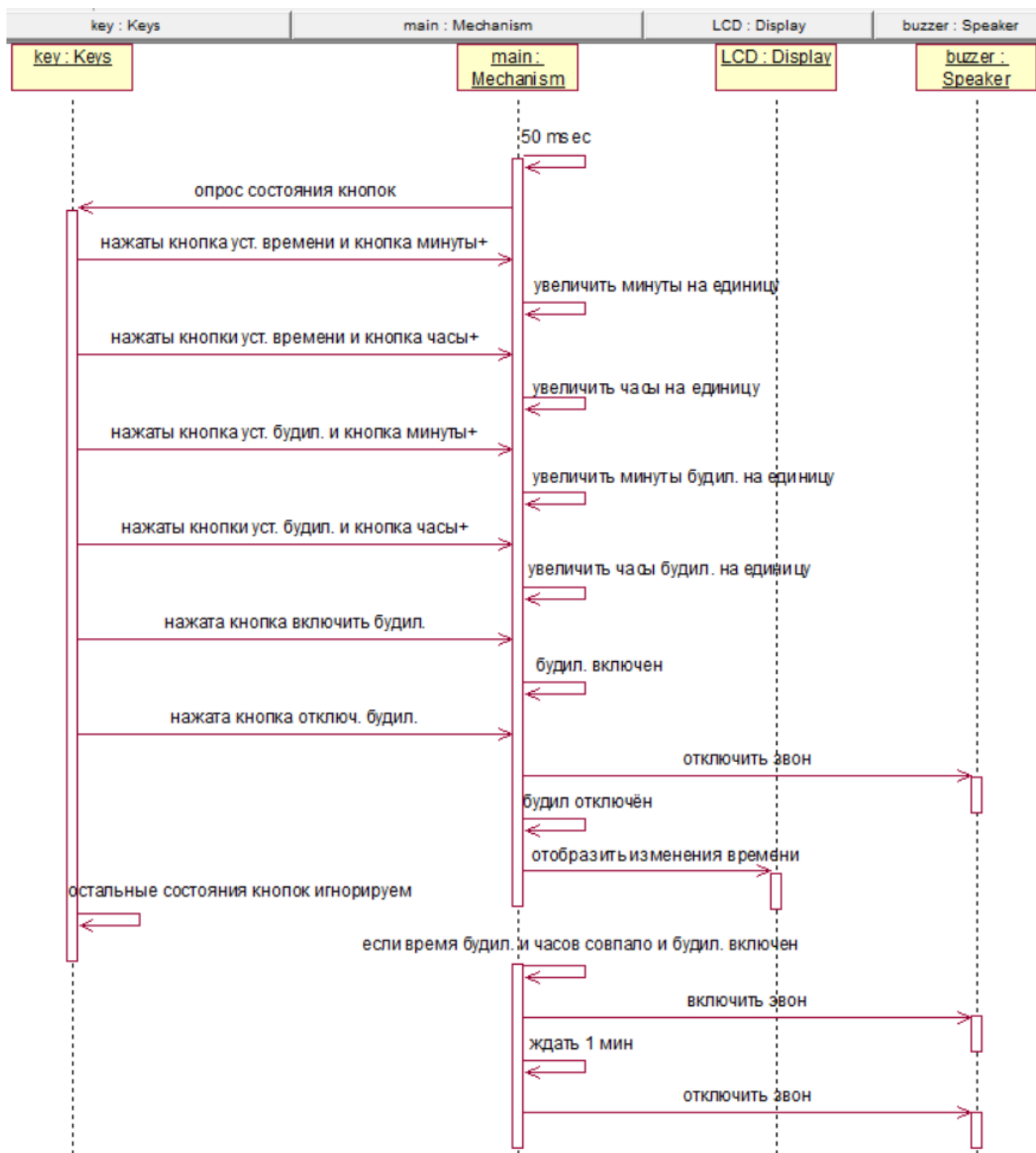


Рисунок 5 – Диаграмма последовательности взаимодействия цикла работы часов

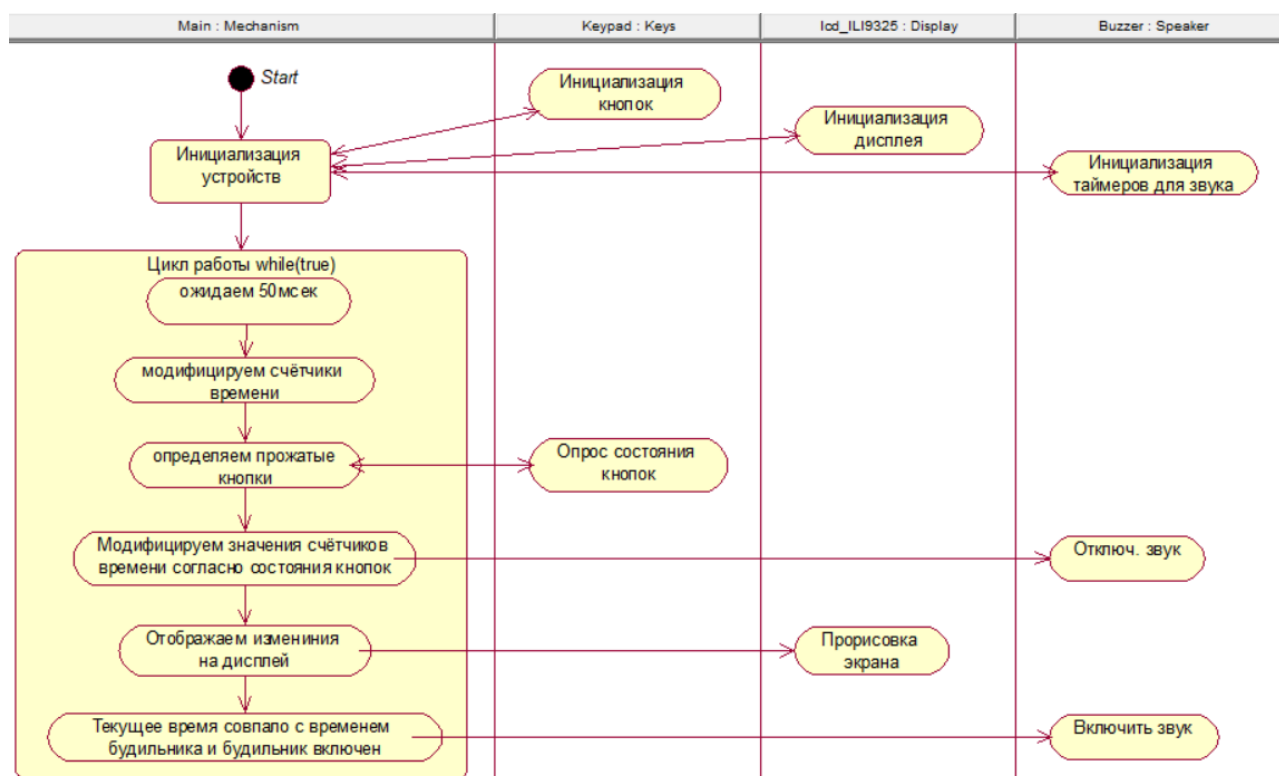


Рисунок 6 – Диаграмма активности функции main

4 Порядок работы с Proteus 8

Proteus представляет собой пакет программного обеспечения для автоматизированного проектирования (САПР) электронных схем на основе моделей электронных компонентов.

Демо версия доступна на официальном сайте: <https://www.labcenter.com/free-trial/>. Либо, на свой страх и риск, со сторонних сайтов: <https://rsload.net/soft/editor/29008-proteus-professional.html>.

Пакет Proteus состоит из двух программных инструментов: ISIS – программа синтеза и моделирования непосредственно электронных схем и ARES – программа разработки печатных плат. Также в состав Proteus восьмой версии входит среда разработки VSM Studio, позволяющая быстро написать программу для микроконтроллера, используемого в проекте, и скомпилировать ее.

Пакет программного обеспечения Proteus имеет следующие особенности:

- моделирование схем с использованием разнообразных виртуальных компонентов в том числе и микроконтроллеров от разных производителей;
- разработка печатной платы, включая 3D-визуализацию ее сборки;
- сквозная разработка электронного устройства от печатной платы до программного обеспечения в единой среде.

Продemonстрируем порядок работы в программе Proteus на основе программы раздела №3 практической работы (общие принципы программного управления микроконтроллером). Продублируем здесь код переключения состояния линии PC13:

```
#include "RTE_Components.h"
#include CMSIS_device_header
void delay(){
    volatile uint32_t count=55370;
    while(count--){
        __NOP();
    }
}
int main(){//управляем PC13
    //Включаем тактирование порта GPIOC
    *(uint32_t*) (0x40021018) |= 0x00000010;
    RCC->APB2ENR|= RCC_APB2ENR_IOPCEN;
    // Настраиваем PC13 как выход
    // Сбрасываем в ноль биты управления 13 выводом
    *(uint32_t*) (0x40011004) &= 0xFF0FFFFF;
    GPIOC->CRH &= ~(GPIO_CRH_MODE13 | GPIO_CRH_CNF13);
    //MODE: выход с частотой 2 МГц
    //CNF: режим push-pull
    *(uint32_t*) (0x40011004) |= 0x00200000;
    SET_BIT(GPIOC->CRH,GPIO_CRH_MODE13_1);
    for(;;){
        *(uint32_t*) (0x40011010) = 0x00002000;
        GPIOC->BSRR = GPIO_BSRR_BS13;
        delay();
        *(uint32_t*) (0x40011014) = 0x00002000;
        GPIOC->BRR = GPIO_BRR_BR13;
        delay();
    }
}
```

Промоделируем работу исполняемого кода микроконтроллером STM32F103C8T6 в Proteus, для этого соберём схему, показанную на рисунке 7.

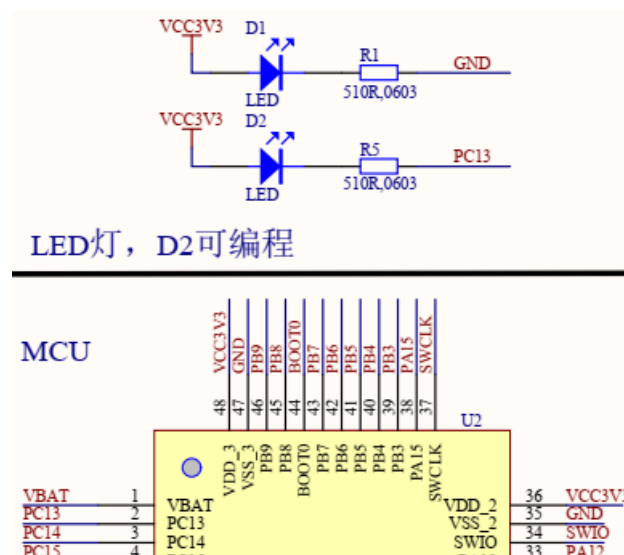


Рисунок 7 – Схема отладочной платы «Blue Pill» в части подключения светодиода D2 LED

Откроем Proteus и создадим проект File >> New Project. В окне New Project Wizard выбираем папку для проекта и имя проекта. В следующем окне выберем Create a schematic from the selected template и в качестве Design Templates выберем Landscape A4. Нажмем Next трижды, а затем Finish.

После создания проекта добавим к контроллеру необходимые элементы (питание, землю, диоды, резисторы и т. д.) и соединим их. Для добавления элементов выбираем в меню Library >> Pick Parts. В поле Keywords вбиваем тип МК stm32f103c8 и в поле Showing local results: выбираем STM32F103C8 и нажимаем OK. Аналогичным образом добавляем в проект светодиод LED-GREEN и сопротивление MINRES510R.

Используя панель инструментов слева, кнопки Component mode и Terminals mode (см. рис. 8), соберём схему показанную на рисунке 9.

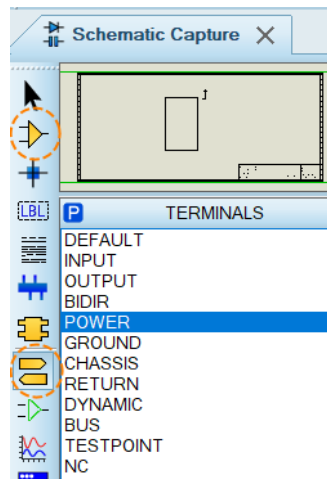


Рисунок 8 – Расположение кнопок Component mode и Terminals mode

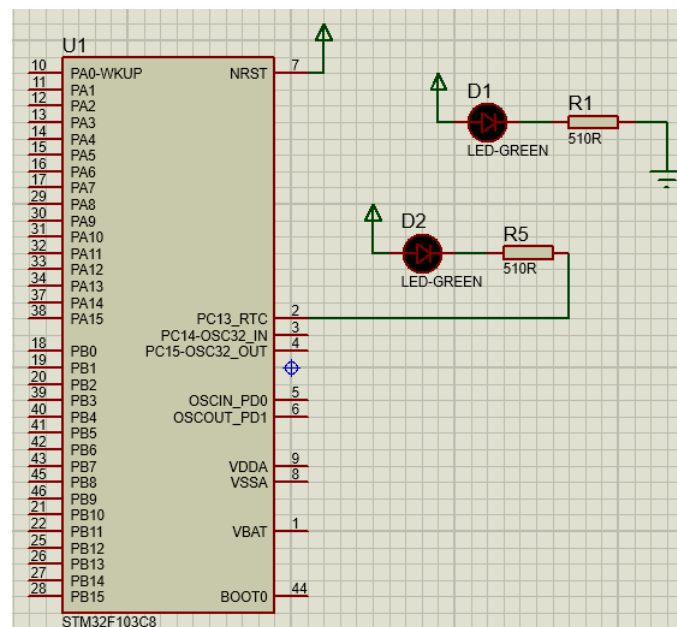


Рисунок 9 – Схема после добавления светодиодов

После того как собрали схему, назначим контроллеру файл с исполняемым кодом, для этого дважды нажмем на STM32F103C8 и укажем путь к файлу в поле Program File (см.

рис. 10). Можно указать файл с расширением .axf (./Lab_kurs_STM32F103x8/3_GPIO/Objects/GPIO.axf), либо файл с расширением .hex, которые получаются в результате компиляции проекта GPIO.uvprojx.

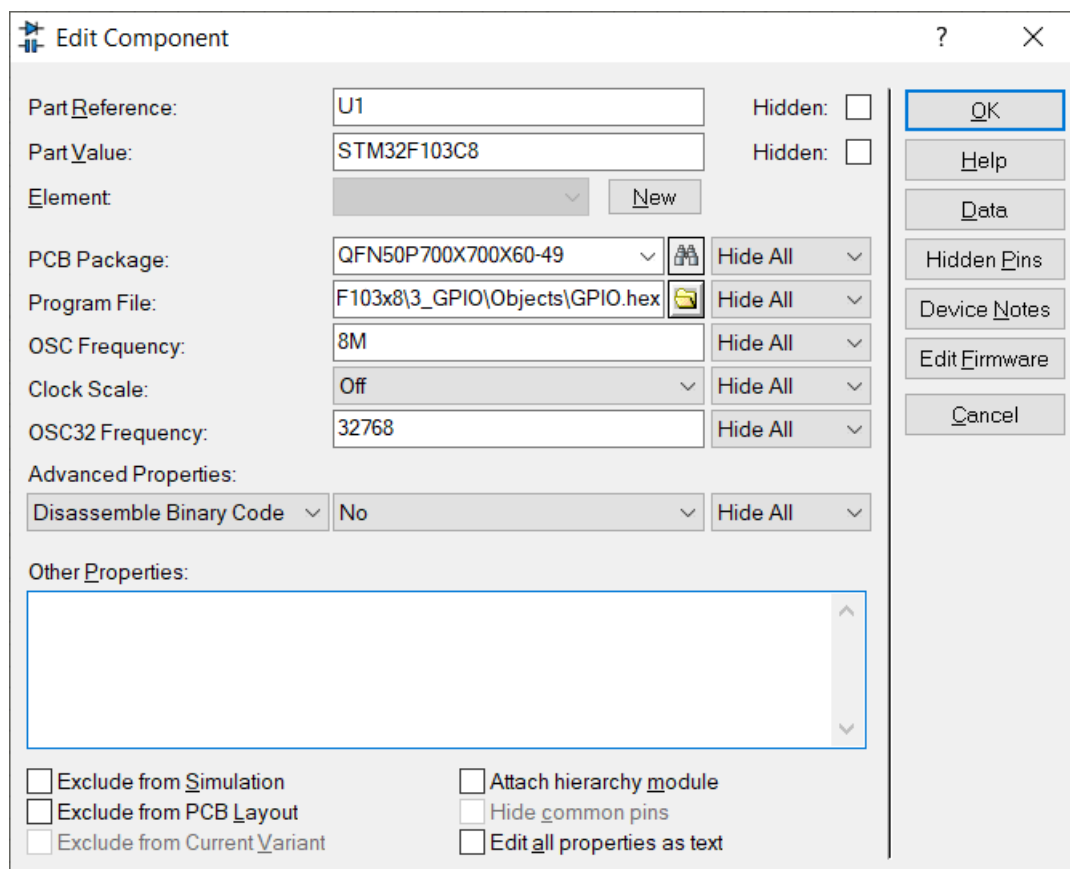


Рисунок 10 – Настраиваемые параметры контроллера

Для компиляции в среде Keil-MDK файла «прошивки» в формате hex необходимо отметить опцию Create HEX File, как показано на рисунке 11.

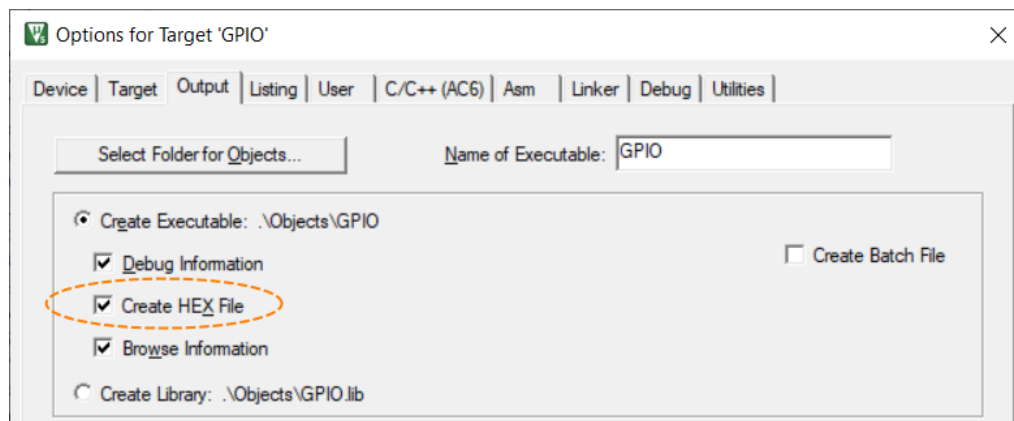


Рисунок 11 – Опция Create HEX File в среде Keil-MDK

Зададим напряжение питания VCC равным 3.3 вольт, выбрать в меню Design >> Configure Power Rails... Далее в окне Power Rail Configuration на вкладке Power Rails в выпадающем поле Power Supplies::Name выбираем VCC/VDD и в поле Voltage вводим 3.3, кнопкой ОК закрываем окно. Для более яркого свечения диодов изменим прямой ток светодиодов: дважды нажмем на светодиод D2, изменим поле Full drive current: на 2mA. Аналогично изменить для светодиода D1.

Далее запустим симуляцию и проверим работу контроллера. Для запуска симуляции необходимо нажать Debug >> Run Simulation (или клавиша F12). После запуска симуляции светодиод D1 (индикатор питания) загорится сразу, а светодиод D2 будет мигать как показано на рисунке 12.

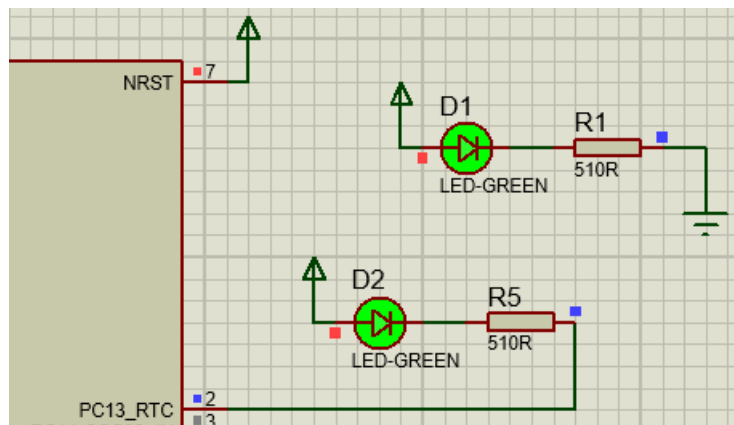


Рисунок 12 – Пример работы контроллера

Для более детального понимания работы с Proteus используйте примеры File >> Open Sample Project и в поле Keywords введите stm32, в таблице Results примеры. Самостоятельно разберитесь как работать с кнопками (Arduino-STM32 Bluepill Button).

4.1 Взаимодействие модели в Proteus со средой разработки Processing посредством интерфейса UART

Внесём изменения в программу, добавив управление включением и выключением светодиода посредством команд, получаемых по интерфейсу UART.

Код программы:

```
#include "RTE_Components.h"
#include CMSIS_device_header

void DMA1_Channel4_IRQHandler(void)
{
    DMA1->IFCR = DMA_IFCR_CTCIF4;
    DMA1_Channel4->CCR &= ~DMA_CCR4_EN;
}

uint16_t length(const char* s)
```



```

{
    uint16_t n = 0;
    for(;*s;s++,n++);
    return n;
}

void sendstr(const char* s)
{
    // wait until DMA channel 4 is disabled for configuration
    while(DMA1_Channel4->CCR&DMA_CCR4_EN);
    DMA1_Channel4->CNDTR = length(s);
    DMA1_Channel4->CPAR = (uint32_t)&USART1->DR;
    DMA1_Channel4->CMAR = (uint32_t)s;
    DMA1_Channel4->CCR = DMA_CCR4_EN | DMA_CCR4_TCIE | DMA_CCR4_MINC | DMA_CCR4_DIR;
    // 8-bit transfer, memory increment, mem2periph
}

uint8_t getc(uint8_t sync)
{
    if (!sync && (USART1->SR&USART_SR_RXNE)==0)
        return 0;
    while((USART1->SR & USART_SR_RXNE)==0);
    return USART1->DR;
}

int main(void)
{
    SystemCoreClockUpdate();
    // turn on peripherals
    RCC->AHBENR |= RCC_AHBENR_DMA1EN; // turn on DMA1 clocking
    RCC->APB2ENR |= RCC_APB2ENR_USART1EN | RCC_APB2ENR_IOPAEN | RCC_APB2ENR_IOPCEN;
    // configure USART1 pins RX-PA10, TX-PA9
    GPIOA->CRH &= ~(GPIO_CRH_MODE9 | GPIO_CRH_CNF9 | GPIO_CRH_MODE10 | GPIO_CRH_CNF10);
    GPIOA->CRH |= GPIO_CRH_MODE9 | GPIO_CRH_CNF9_1 | GPIO_CRH_CNF10_1;
    GPIOA->BSRR = GPIO_ODR_ODR10;
    // configure USART1
    // USARTDIV = Fck / (16 * BAUD) = 8000000 / (16 * 115200) = 4.34
    // DIV_Fraction = 16*0d0.34 = 0d5,44 = 0x5
    USART1->BRR = (4<<4) + 5; // mantissa = 4, fractional = 5
    // Parity Control (disabled), Receiver and Transmitter(enabled)
    USART1->CR1 = USART_CR1_UE | USART_CR1_RE | USART_CR1_TE;
    USART1->CR3 = USART_CR3_DMAT;
    NVIC_EnableIRQ(DMA1_Channel4_IRQn);
    NVIC_EnableIRQ(USART1_IRQn);
    sendstr("\r\nUsart on\r\n");

    //Led PC13
    GPIOC->CRH &= ~(GPIO_CRH_MODE13 | GPIO_CRH_CNF13);
    SET_BIT(GPIOC->CRH,GPIO_CRH_MODE13_1);
    GPIOC->BSRR = GPIO_BSRR_BS13;
    sendstr("Led off\r\n");
    for(;;)
    {
        char rbyte = getc(0);
        switch(rbyte)
        {
            case 'o':
                GPIOC->BRR = GPIO_BRR_BR13; //on
                sendstr("Led light\r\n");
                break;
            case 'p':
                GPIOC->BSRR = GPIO_BSRR_BS13; //off
                sendstr("Led dark\r\n");
                break;
            default:
                if(rbyte)
                {
                    while((USART1->SR & USART_SR_TXE)==0);
                    USART1->DR = rbyte;
                }
                break;
        }
    }
}

```

В программе обмен по интерфейсу UART осуществляется на скорости 115200 бит/с. Для передачи используется контроллер прямого доступа к памяти DMA1, канал номер четыре. Приём осуществляется асинхронно байтами функцией `getc(0)`.

Проверим работу программы в Proteus. Добавим к схеме рисунка 9 элемент Virtual Terminal: на панели инструментов слева, кнопка Virtual Instruments Mode, показана на рисунке 13.

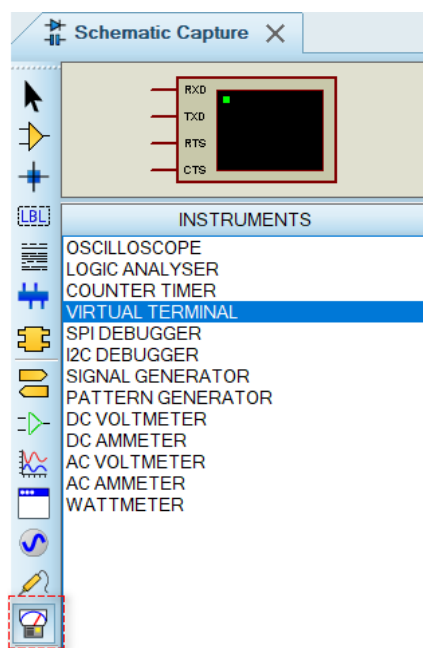


Рисунок 13 – Расположение элемент Virtual Terminal

Подсоединим терминал к выводам UART1 PA9(линия передачи) и PA10(линия приёма), как показано на рисунке 14. В свойствах терминала измените скорость обмена на 115200 бит/с, как показано на рисунке 15.

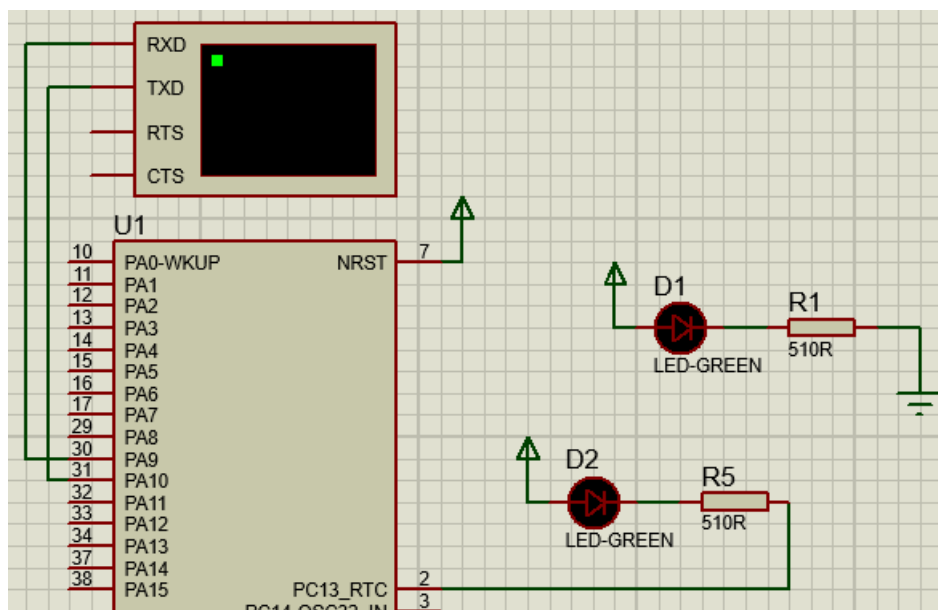


Рисунок 14 – Схема после добавления элемента Virtual Terminal

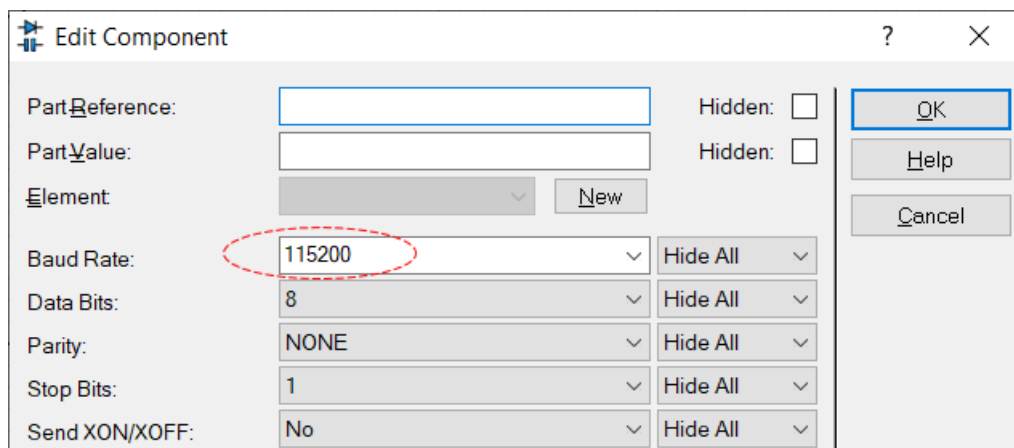


Рисунок 15 – Свойства компонента Virtual Terminal

После запуска моделирования откроется окно Virtual Terminal и в нём будут напечатаны две строки – “Usart on” и “Led off”. Если окна нет или оно было закрыто ранее, то через меню Debug >> Virtual Terminal выведите снова. Если выведен непонятный набор символов, значит скорость обмена терминала настроена не верно. Выделите окно терминала и нажатием кнопок убедитесь в работе программы: символ ‘o’ включает светодиод D2; символ ‘p’ выключает.

После того как мы убедились в работоспособности интерфейса UART, удаляем Virtual Terminal. И подключаем виртуальный порт: Library >> Pick Parts, в поле Keywords вбиваем COMPM, получаем схему показанную на рисунке 16.

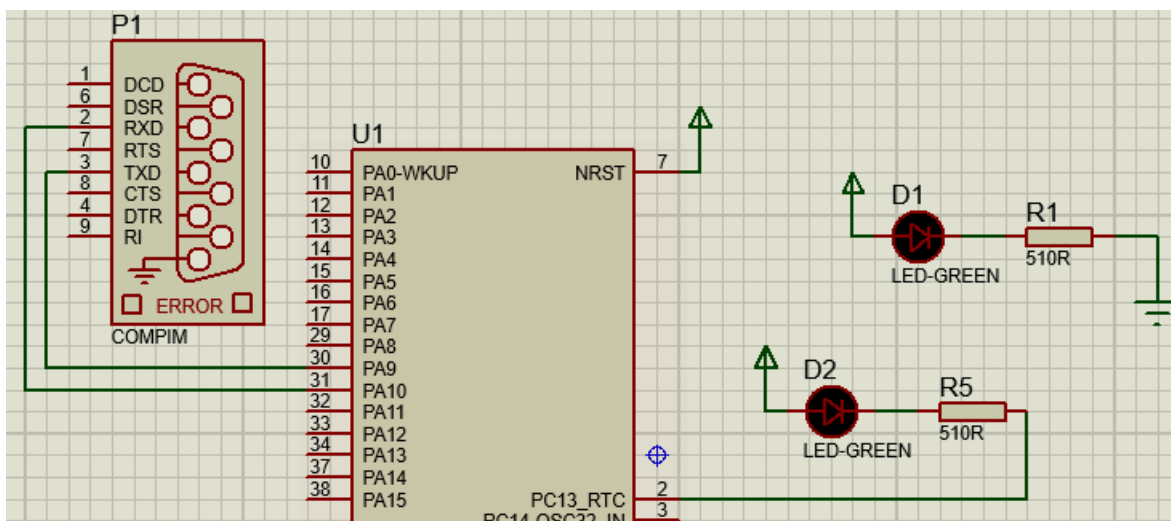


Рисунок 16 – Схема после добавления элемента COMPIM

Далее нам понадобится еще две программы. Одна для создания виртуального соединения из двух COM портов: Virtual Serial Port Driver Pro, пробную версию можно скачать с официального сайте <https://www.eltima.com/ru/products/vspdxp/>. А вторая, терминал для работы с последовательным портом - Termite, с сайта https://www.compuphase.com/software_termite.htm.

Сначала устанавливаем первую программу для эмуляции пары: виртуальный порт <<>> виртуальный порт. После добавления пары связанных com портов (см. рис. 17) COM3<<>>COM4, в свойствах COMPIM устанавливаем порт COM3 и скорость работы интерфейса: 115200 бит/с (см.рис. 18).

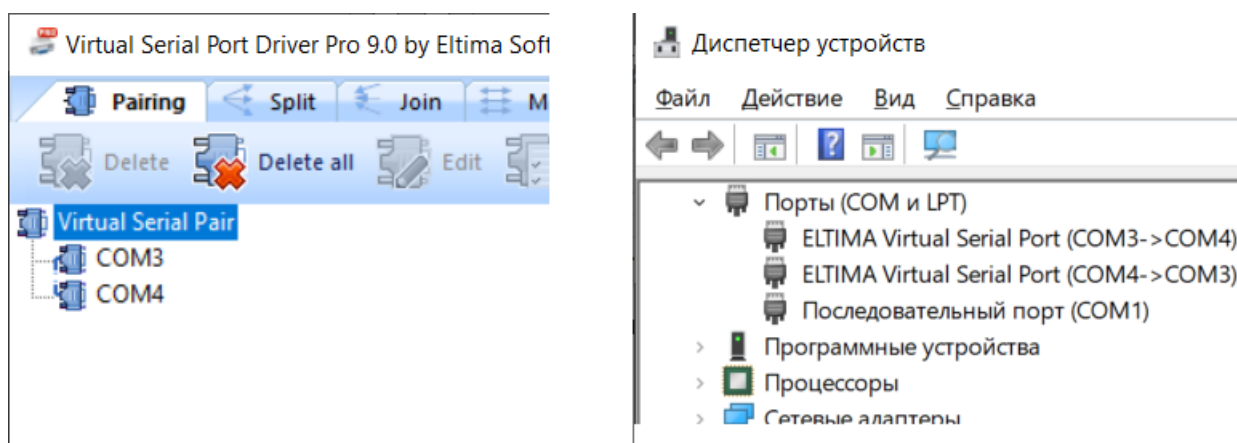


Рисунок 17 – Добавление пары связанных виртуальных портов в ОС Windows

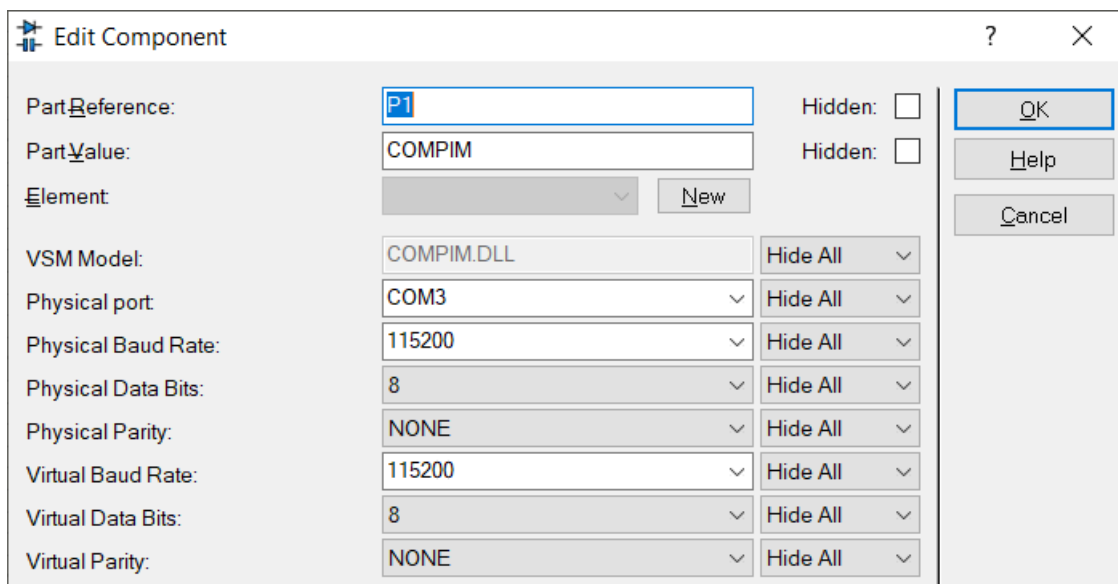


Рисунок 18 – Свойства компонента COMPIM

В настройка терминала Terminate устанавливаем порт COM4 и скорость 115200, как показано на рисунке 19.

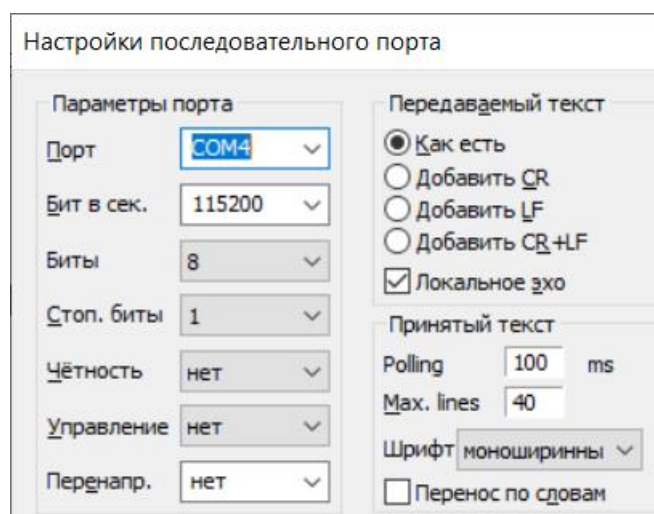


Рисунок 19 – Настройка последовательного порта в программе Terminate

После запуска симуляции в Proteus, на терминале Terminate будет выведено две строки. По нажатию ввода из терминала Terminate будет отправлен введенный в нижней строке символ в порт COM4, далее в сопряжённый порт COM3, далее посредством COMPIM символ будет передан в периферийное устройство UART микроконтроллера, и светодиод изменит состояние, как показано на рисунке 20.

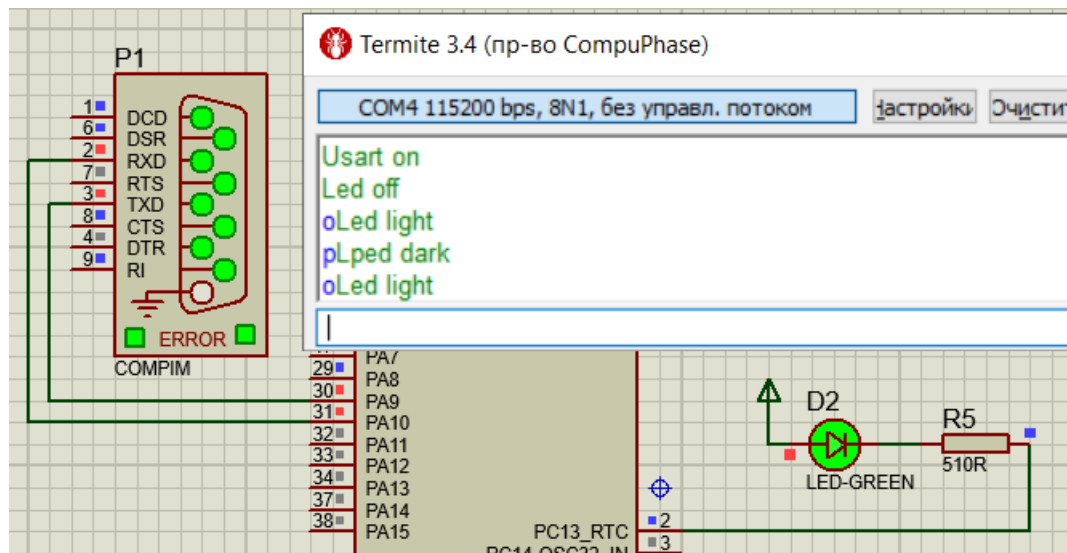


Рисунок 20 – Управление светодиодом в Proteus посредством терминала Termite

В качестве терминала можно использовать и специальные программы, например свободно распространяемый клиент для различных протоколов удалённого доступа PuTTY (см. рис. 21), но для курсовой работы используем среду разработки Processing, которая свободно скачивается по адресу: <https://processing.org/download>. В нашем случае Processing позволяет создавать прототип интерфейса управляющей части, где в качестве исполнителя выступает микроконтроллер.

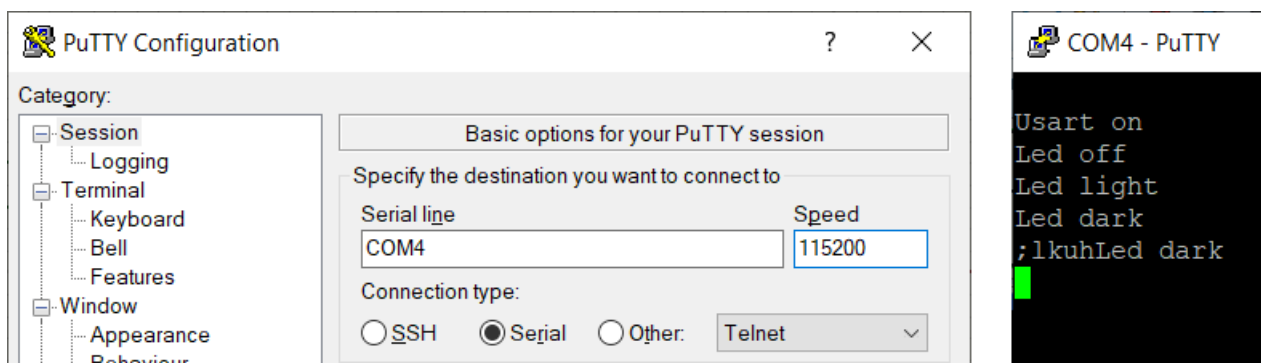


Рисунок 21 – Настройка последовательного порта в программе PuTTY

Воспользуемся примером Button расположенным по адресу: <https://processing.org/examples/button.html> и примером функции чтения com порта из библиотеки Serial: https://processing.org/reference/libraries/serial/Serial_readChar_.html.

Код программы в Processing:

```
import processing.serial.*;
Serial myPort; // The serial port:
String recvstr="";

int circleX, circleY; // Position of circle button
int circleSize = 113; // Diameter of circle
```

```

color circDark, circleHighlight, currentColor;
boolean circleOver = false;
boolean ledStatus = false;

void setup() {
    size(640, 360);
    background(0);
    textFont(createFont("Consolas", 18));
    circDark = color(10,80,10);
    circleHighlight = color(0,255,0);
    currentColor = circDark;
    circleX = width/2;
    circleY = height/2;
    ellipseMode(CENTER);
    myPort = new Serial(this, "COM4", 115200);
}

void draw() {
    float disX = circleX - mouseX;
    float disY = circleY - mouseY;
    if (sqrt(sq(disX) + sq(disY)) < circleSize/2 )
        circleOver = true;
    else
        circleOver = false;
    fill(currentColor);
    ellipse(circleX, circleY, circleSize, circleSize);
    if (myPort.available() > 0) {
        char inByte = myPort.readChar();
        recvstr = recvstr + str(inByte);
        print(inByte);
        if ( inByte == '\n')
        {
            recvstr = recvstr + str('\0');
            fill(0);
            rect(0,0,200,30);
            fill(255);
            text(recvstr,20,20);
            recvstr = "";
        }
    }
}

void mousePressed() {
    if (circleOver)
    {
        if(ledStatus) {
            myPort.write('o');
            currentColor = circleHighlight;
            ledStatus = false;
        }
        else {
            myPort.write('p');
            currentColor = circDark;
            ledStatus = true;
        }
    }
}

```

После запуска программы в Processing (эмуляция в Proteus также должна быть запущена) нажмите левой кнопкой мышки на тёмно-зелёный круг в центре появившегося окна. При прожатии состояние круга, как и светодиода в Proteus будет попеременно меняться как показано на рисунке 22. Таким образом через канал связи UART реализуется управление микроконтроллером в Proteus.

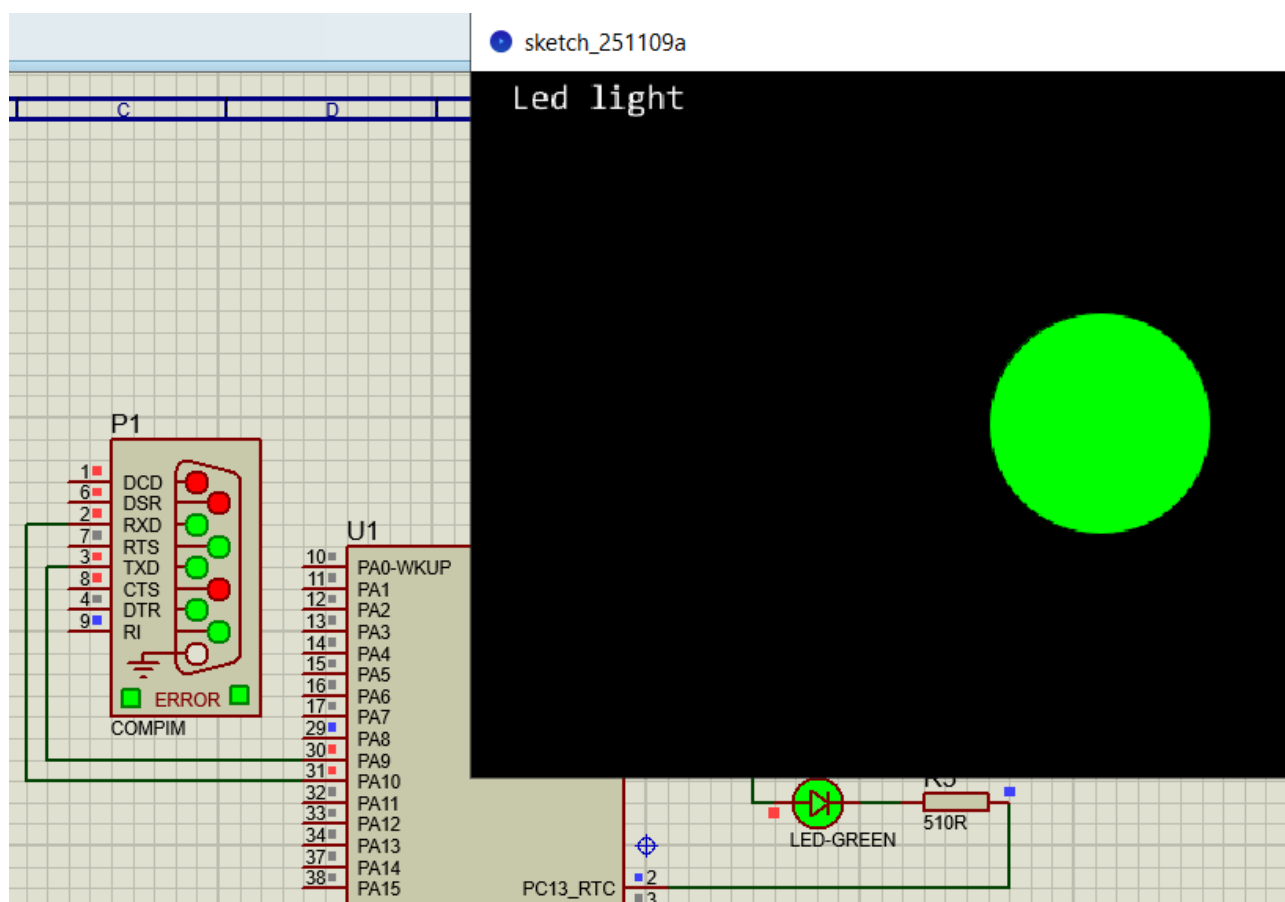


Рисунок 22 – Управление светодиодами в Proteus посредством программы в Processing

4.2 Порядок подключения LCD экрана в Proteus на примере часов RTC

Подключим к микроконтроллеру дисплей LM016L [1] по схеме показанной на рисунке 23.

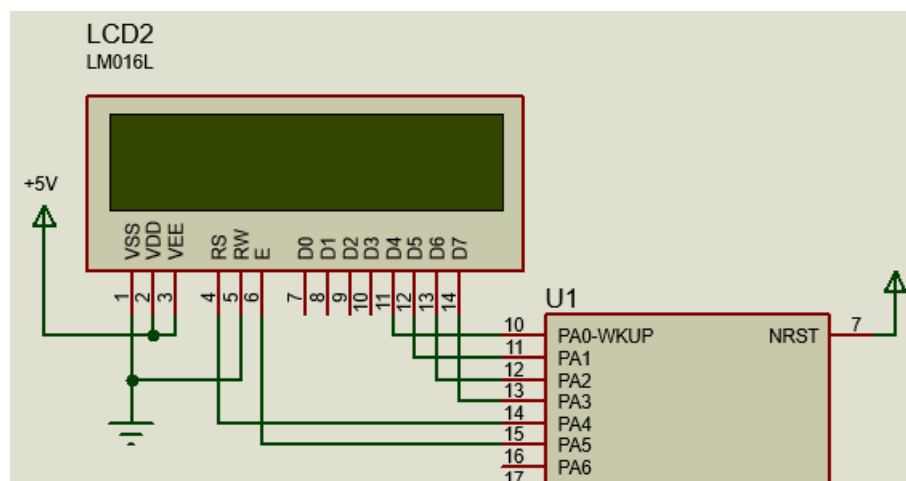


Рисунок 23 – Схема подключения LCD LM016L к STM32F103C8

Создадим в Keil проект с подключение библиотеки HAL LL. Программа часов с использованием модуля RTC (часы реального времени), файл main.c:

```
#include "stm32f1xx_ll_bus.h"
#include "stm32f1xx_ll_rcc.h"
#include "stm32f1xx_ll_utils.h"
#include "stm32f1xx_ll_gpio.h"
#include "stm32f1xx_ll_rtc.h"
#include "stm32f1xx_ll_pwr.h"
#include <stdio.h>
#include "LCD_LM016L.h"

void outputTime();
static uint8_t outputFlag = 0;
static uint32_t timeCounter = 0;

int main(void)
{
    SystemCoreClockUpdate();
    LL_Init1msTick(SystemCoreClock);
    printf("Core clk=%d Hz\n",SystemCoreClock);
    LCD_Init();
    //PC13 - LED
    LL_APB2_GRP1_EnableClock(LL_APB2_GRP1_PERIPH_GPIOC);
    LL_GPIO_SetPinMode(GPIOC, LL_GPIO_PIN_13, LL_GPIO_MODE_OUTPUT);
    //-----RTC configuration-----
    //Подключение LSE к RTC
    LL_APB1_GRP1_EnableClock(LL_APB1_GRP1_PERIPH_PWR); //включение power interface
    LL_PWR_EnableBkUpAccess(); //отключение защиты записи регистров RTC и Backup: PWR_CR_DBP >> 1

    LL_RCC_LSE_Enable(); //включение LSE => RCC_BDCR_LSEON
    while(!LL_RCC_LSE_IsReady()) {} //Ожидание готовности LSE => RCC_BDCR_LSERDY
    //выбор источника для RTC - кварцовый осциллятор LSE 32768Hz
    LL_RCC_SetRTCClockSource(LL_RCC_RTC_CLKSOURCE_LSE);
    LL_RCC_EnableRTC(); //включение RTC => RCC_BDCR_RTCEN

    LL_RTC_DisableWriteProtection(RTC);
    if(LL_RTC_EnterInitMode(RTC) == ERROR) while(1){}; // ERROR enter init mode rtc
    //установить prescaler => RTC_PRL, получение сигнала с периодом в 1 секунду
    LL_RTC_SetAsynchPrescaler(RTC, 0x7FFF);
    LL_RTC_TIME_Set(RTC, 0xFFFFFFFF0); // RTC_CNT
    /* Wait until last write operation on RTC registers has finished */
    LL_RTC_WaitForSynchro(RTC);
    LL_RTC_EnableIT_SEC(RTC);
    LL_RTC_ExitInitMode(RTC); //выход из режима настройки => RTC_CRL_RTOFF RTC_CRL_CNF
    LL_RTC_EnableWriteProtection(RTC);
    NVIC_EnableIRQ(RTC_IRQn);
    LCD_ClearScreen();
    while(1) {
        if(outputFlag) {
            outputFlag=0;
            outputTime();
        }
    }

    //-----Outputs-----
    void outputTime() {
        uint32_t day, hour, min, sec;
        printf("\33[0d\33[2J"); //clear debug screen
        day = timeCounter/(24*3600);
        hour = (timeCounter/3600) % 24;
        min = (timeCounter % 3600) / 60;
        sec = (timeCounter % 3600) % 60;
        char buf[17];
        sprintf(buf,"day = %0.10d",day);
        printf("%s ",buf);
        LCD_Cursor(0,0);
        LCD_PrintStr(buf);
        sprintf(buf,"%0.2d:%0.2d:%0.2d",hour,min,sec);
        printf("%s\n",buf);
        LCD_Cursor(1,0);
        LCD_PrintStr(buf);
    }

    //-----RTC Handler-----
    void RTC_IRQHandler(void) {
```

```

LL_GPIO_TogglePin(GPIOC, LL_GPIO_PIN_13);
if(LL_RTC_IsActiveFlag_SEC(RTC))
{
    timeCounter = LL_RTC_TIME_Get(RTC);
    LL_RTC_ClearFlag_SEC(RTC);
    outputFlag= 1;
}
}

```

Листинг заголовочного файла LCD_LM016L.h:

```

#ifndef LCD_LM016L
#define LCD_LM016L

void LCD_Init(void);
void LCD_Cursor(char Row, char Col);
void LCD_ClearScreen(void);
void LCD_PrintStr(char *Text);

#endif

```

Листинг файла реализации интерфейса взаимодействия с дисплеем, LCD_LM016L.c:

```

#include "LCD_LM016L.h"
#include "stm32f1xx_ll_bus.h"
#include "stm32f1xx_ll_utils.h"
#include "stm32f1xx_ll_gpio.h"

#define LCD_OUT          GPIOA->ODR
#define LCD_PIN_EN       LL_GPIO_PIN_5
#define LCD_PIN_RS       LL_GPIO_PIN_4
#define LCD_PIN_D7       LL_GPIO_PIN_3
#define LCD_PIN_D6       LL_GPIO_PIN_2
#define LCD_PIN_D5       LL_GPIO_PIN_1
#define LCD_PIN_D4       LL_GPIO_PIN_0
#define LCD_PIN_MASK     ((LCD_PIN_RS|LCD_PIN_EN|LCD_PIN_D7|LCD_PIN_D6|LCD_PIN_D5|LCD_PIN_D4))

void LCD_strob(void);
void LCD_SendByte(char ByteToSend, int IsData);

void LCD_strob()
{
    LCD_OUT &= ~LCD_PIN_EN;
    LL_mDelay(1);
    LCD_OUT |= LCD_PIN_EN;
    LL_mDelay(2);
    LCD_OUT &= (~LCD_PIN_EN);
    LL_mDelay(40);
}

void LCD_SendByte(char ByteToSend, int IsData)
{
    LCD_OUT &= (~LCD_PIN_MASK);
    LCD_OUT |= (ByteToSend & 0xF0)>>4;
    if (IsData == 1)
        LCD_OUT |= LCD_PIN_RS;
    else
        LCD_OUT &= ~LCD_PIN_RS;
    LCD_strob();

    LCD_OUT &= (~LCD_PIN_MASK);
    LCD_OUT |= (ByteToSend & 0x0F);
    if (IsData == 1)
        LCD_OUT |= LCD_PIN_RS;
    else
        LCD_OUT &= ~LCD_PIN_RS;
    LCD_strob();
}

void LCD_Cursor(char Row, char Col)
{
    LCD_SendByte( (((Row)&1)*0x40)+(((Row)&3)>>1)*0x14)+(Col)|128,0);
}

void LCD_ClearScreen()
{
    LCD_SendByte(0x01, 0);
    LCD_SendByte(0x02, 0);
}

```

```

        LCD_SendByte(0x01, 0);
    }
    void LCD_Init(void)
    {
        LL_APB2_GRP1_EnableClock(LL_APB2_GRP1_PERIPH_GPIOA);
        LL_GPIO_SetPinMode(GPIOA, LCD_PIN_EN, LL_GPIO_MODE_OUTPUT);
        LL_GPIO_SetPinMode(GPIOA, LCD_PIN_RS, LL_GPIO_MODE_OUTPUT);
        LL_GPIO_SetPinMode(GPIOA, LCD_PIN_D7, LL_GPIO_MODE_OUTPUT);
        LL_GPIO_SetPinMode(GPIOA, LCD_PIN_D6, LL_GPIO_MODE_OUTPUT);
        LL_GPIO_SetPinMode(GPIOA, LCD_PIN_D5, LL_GPIO_MODE_OUTPUT);
        LL_GPIO_SetPinMode(GPIOA, LCD_PIN_D4, LL_GPIO_MODE_OUTPUT);

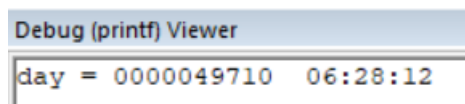
        LCD_SendByte(0x02, 0);
        LCD_SendByte(0x28, 0);
        LCD_SendByte(0x0C, 0);
        LCD_ClearScreen();

        LCD_Cursor(0,0);
        LCD_PrintStr("Tempus ..."); //Написание текста
        LCD_Cursor(1,0);
        LCD_PrintStr("Tempus advenit.");
        LCD_Cursor(1,2);
        LCD_PrintStr("Te");
    }

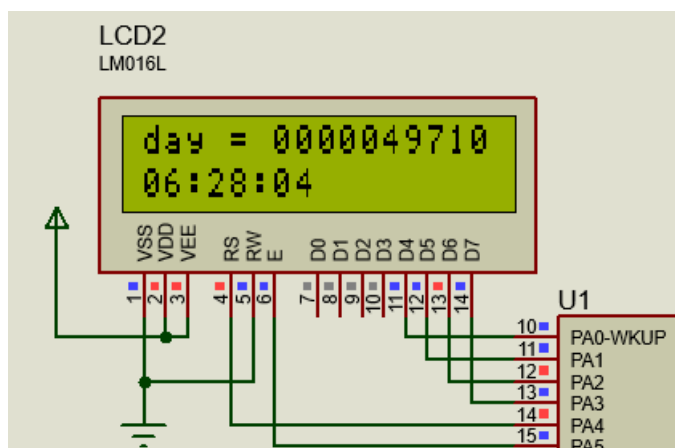
    void LCD_PrintStr(char *Text)
    {
        char *c;
        c = Text;
        while ((c != 0) && (*c != 0))
        {
            LCD_SendByte(*c, 1);
            c++;
        }
    }
}

```

После компиляции убедитесь в симуляции Keil в правильности работы приложения. В окне Debug (printf) Viewer должен быть выведен день и часы:



Далее подключите полученный загрузочный файл .axf к микроконтроллеру в проекте Proteus и при запуске симуляции на экране будут выведены день и часы:



Здесь нужно отметить, что файлы, реализующие интерфейс с LCD (LCD_LM016L.h/c) не будут работать на реальном экране, поскольку для экономии размера кода временные задержки при инициализации модуля LCD были сокращены.

Подобным образом можно подключить экран LM041L на 4 строки [2], с чем в процессе работы над курсовым проектом предлагается разобраться самостоятельно.

5 Порядок работы с симулятором Keil

Рассмотрим на примере реализации опроса состояния кнопок, в продолжение раздела 3. Определим состояния, в которых может находиться кнопка и переходы между ними.

Возможные состояния кнопки:

- Key_init – начальная инициализация переменных состояния кнопок;
- Key_up - кнопка отжата, состояние по умолчанию;
- Key_down – кнопка нажата;
- Key_uptodn – состояние перехода между отжата и нажата, поскольку при замыкании контактов кнопки возникает переходной процесс, то необходимо выдержать некоторое время (менее 1 мс) чтобы сигнал устоялся и только после этого принимать решение о состоянии кнопки;

замыкании контактов кнопки возникает переходной процесс, то необходимо выдержать некоторое время (менее 1 мс) чтобы сигнал устоялся и только после этого принимать решение о состоянии кнопки;

- Key_dntoup - состояние перехода между нажата и отжата, такие же переходные процессы возникают и при размыкании контактов;

- Key_dnlong – состояние удержания нажатой кнопки.

Граф перехода между состояниями показан на рисунке 24.

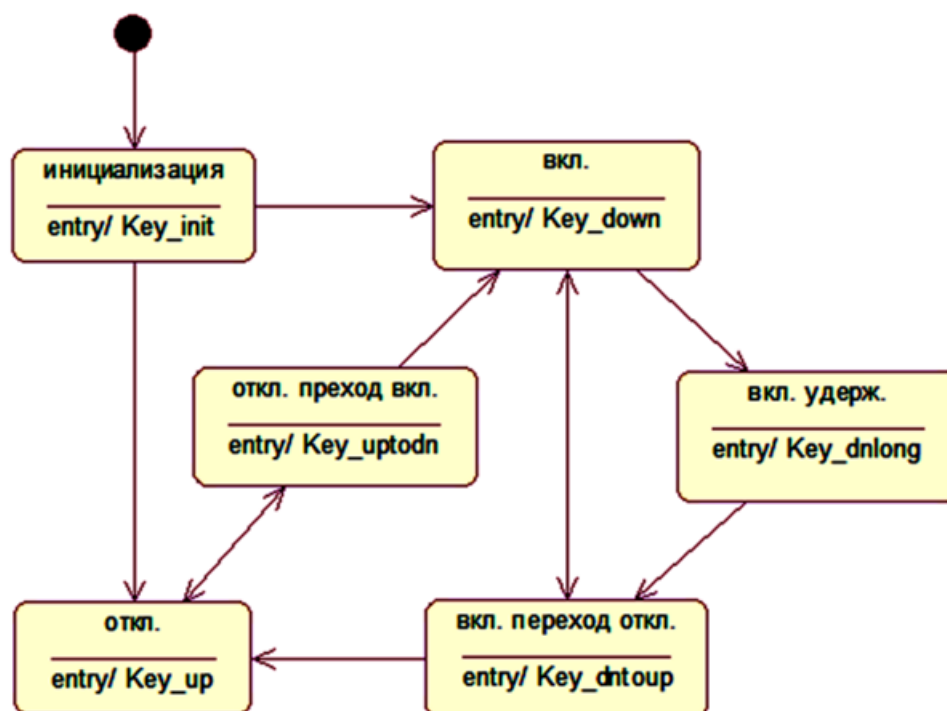


Рисунок 24 – Граф перехода между состояниями кнопки

Переход между состояниями установим 20 миллисекунд, это время и станет периодом цикла опроса кнопок. 200 мсек будет периодом повторения события нажатия для удерживаемой кнопки.

Файл main.c – пример обработки нажатия кнопок с учётом явления дребезга:

https://github.com/vocmok/Simul_guide_Keil_STM32F103X/tree/main/Debounce_filt_sim

```
#include "RTE_Components.h"
#include CMSIS_device_header
#include <stdio.h>
#define BT_TREPEAT 10 //время повторения символа при зажатой кнопке 10*20ms=200ms
typedef enum
{
    Key_init = (0x00), /*начальное*/
    Key_down = (0x01), /*включена*/
    Key_up = (0x02), /*отключена*/
    Key_dntoup = (0x04), /*переход от включена до отключена*/
    Key_uptodn = (0x08), /*переход от отключена до включена*/
    Key_dnlong = (0x10), /*долгое нажатие*/
} KEY_state;
typedef struct
{
    KEY_state state;
    unsigned int downlongtick;
} KEY_type;
typedef enum
{
    Sgnl_NULL = (0x00), /*на выходе ноль*/
    Sgnl_key0 = (0x01), /*нажата кнопки 0*/
    Sgnl_key1 = (0x02), /*нажата кнопки 1*/
} SGNL_state;
typedef struct
{
    SGNL_state state;
    uint8_t flag20ms;
    uint8_t flag1sec;
} ISRdat_type;
__IO ISRdat_type isrdat;
unsigned int keypad_poll(void);
int main(){ //clk 72MHz
    SystemCoreClockUpdate();
    printf("clk=%d Hz\n", SystemCoreClock);
    ITM_SendChar('\n');
    //Включаем тактирование GPIOA, GPIOC
    SET_BIT(RCC->APB2ENR, RCC_APB2ENR_IOPAEN | RCC_APB2ENR_IOPCEN);
    // Настраиваем PA0 и PA1 как вход с подтяжкой к питанию
    // Сбрасываем в ноль биты управления PA0 и PA1 выводом
    GPIOA->CRL &= ~(GPIO_CRL_MODE0 | GPIO_CRL_CNF0 | GPIO_CRL_MODE1 | GPIO_CRL_CNF1 );
    SET_BIT(GPIOA->CRL, GPIO_CRL_CNF0_1|GPIO_CRL_CNF1_1); //PA 0,1 Input, Pull up
    SET_BIT(GPIOA->ODR, GPIO_ODR_ODR0|GPIO_ODR_ODR1); //PA 0,1 Pull up
    GPIOC->CRH &= ~(GPIO_CRH_MODE13 | GPIO_CRH_CNF13);
    SET_BIT(GPIOC->CRH, GPIO_CRH_MODE13_1); //PC13 push-pull 2MHz
    isrdat.state=Sgnl_NULL;
    isrdat.flag1sec=0;
    isrdat.flag20ms=0;
    SysTick_Config(0xAFC80); //прерывание системного таймера каждые 10 мсек
    uint32_t cntsec=0;
    for(;;){
        if(isrdat.flag1sec){
            cntsec++;
            printf("%d sec\n", cntsec); //выводим каждые 1 сек
            isrdat.flag1sec=0;
        }
        if(isrdat.flag20ms){
            uint32_t keystate=keypad_poll();
            if(keystate&0x1)
            {
                printf("Key0 PA0 pressed\n");
                isrdat.state=Sgnl_key0;
            }
            if(keystate&0x2)
            {
                printf("Key1 PA1 pressed\n");
                isrdat.state=Sgnl_key1;
            }
        }
    }
}
```

```

        if(keystate&(0x1<<16))
        {
            printf("Key0 PA0 up\n");
            isrdat.state=Sgnl_NULL;
        }
        if(keystate&(0x2<<16))
        {
            printf("Key1 PA1 up\n");
            isrdat.state=Sgnl_NULL;
        }
        isrdat.flag20ms=0;
    }
}

void SysTick_Handler(void)//обработчик прерывание системного таймера
{
    static uint32_t ui_cnt10ms=0;
    ui_cnt10ms++;
    if(ui_cnt10ms%2==0)
        isrdat.flag20ms=1;
    if(ui_cnt10ms%100==0)
        isrdat.flag1sec=1;
    switch(isrdat.state){
        case Sgnl_NULL:
            GPIOC->ODR = 0;
            break;
        case Sgnl_key0://частота 50 Hz
            GPIOC->ODR ^= GPIO_ODR_ODR13;
            break;
        case Sgnl_key1://частота 10 Hz
            if(ui_cnt10ms%5==0)
                GPIOC->ODR ^= GPIO_ODR_ODR13;
            break;
    }
}

unsigned int keypad_poll()
{
    static KEY_type masKey[2];
    static uint32_t kpd_pollcnt=0;
    kpd_pollcnt++; //счётчик кол-ва опросов
    uint32_t retcode=0;
    KEY_type* pcurKey;
    for(int i=0;i<2;i++) { /*опрос состояния линий (кнопок)*/
        pcurKey=&masKey[i];
        if(GPIOA->IDR&(1<<i)) { /*ур на линии единица значит не нажата*/
            switch(pcurKey->state) {
                case Key_dntoup:
                    retcode|=1<<(i+16); //возвращаем флаг отжатия кнопки в старших 16 битах
                case Key_init:
                case Key_up:
                case Key_uptodn:
                    pcurKey->state=Key_up;
                    break;
                case Key_down:
                case Key_dnlong:
                    pcurKey->state=Key_dntoup;
                    break;
            }
        }
        else { /*down на линии ноль значит нажата*/
            switch(pcurKey->state) {
                case Key_init:
                case Key_dntoup:
                    pcurKey->state=Key_down;
                    break;
                case Key_up:
                    pcurKey->state=Key_uptodn;
                    break;
                case Key_uptodn:
                    pcurKey->state=Key_down;
                    retcode|=1<<i; //возвращаем флаг нажатия кнопки в младших 16 битах
                    break;
                case Key_down:
                    pcurKey->state=Key_dnlong;
                    pcurKey->downlongtick=kpd_pollcnt;
                    break;
                case Key_dnlong:
            }
        }
    }
}

```

```

        if((kpd_pollcnt-pcurKey->downlongtick)%BT_TREPEAT==0)
            retcode|=1<<i;//повторно возвращаем флаг нажатия кнопки (зажата)
        break;
    }
}
return retcode;
}

```

Файл GPIOA_But.ini (скрипт создания двух кнопок на панели инструментов для имитации поведения при нажатии реальных кнопок):

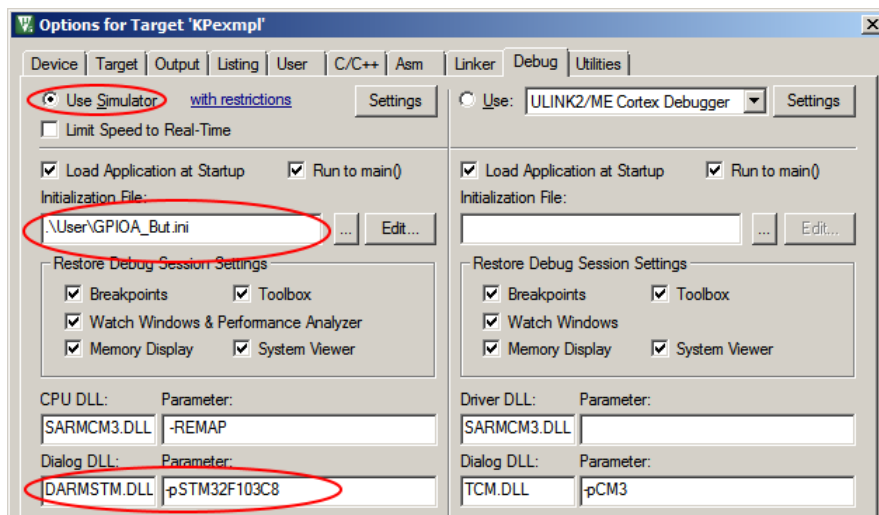
```

//KILL BUTTON 1 //удалить старые кнопки
//KILL BUTTON 1 //два раза подряд (две кнопки)
OSC = 8000000; //Установили частоту внешнего генератор Xtal (Hz)
PORTA |= 0x03;
SIGNAL void toggle_A0_pin() {
    int i;
    for(i=0; i<10;i++){//имитация дребезга при нажатии
        PORTA = (PORTA ^ 0x01);
        swatch (0.002);} /* ждать 2 мс */
    PORTA &= ~0x01; //установившееся значение ноль
    swatch (0.5); /* удержание кнопки 0.5s */
    for(i=0; i<10;i++){//имитация дребезга при отпускании
        PORTA = (PORTA ^ 0x01);
        swatch (0.002);} /* ждать 2 мс */
    PORTA |= 0x01;//установившееся значение единица
}
SIGNAL void toggle_A1_pin() {
    int i;
    for(i=0; i<10;i++){//имитация дребезга при нажатии
        PORTA = (PORTA ^ 0x02);
        swatch (0.002);} /* ждать 2 мс */
    PORTA &= ~0x02; //установившееся значение ноль
    swatch (0.5); /* удержание кнопки 0.5s */
    for(i=0; i<10;i++){//имитация дребезга при отпускании
        PORTA = (PORTA ^ 0x02);
        swatch (0.002);} /* ждать 2 мс */
    PORTA |= 0x02;//установившееся значение единица
}
//создать кнопку Key0 PA0 и привязать к ней функцию toggle_A0_pin()
DEFINE BUTTON "Key0 PA0", "toggle_A0_pin()"
DEFINE BUTTON "Key1 PA1", "toggle_A1_pin()"

```

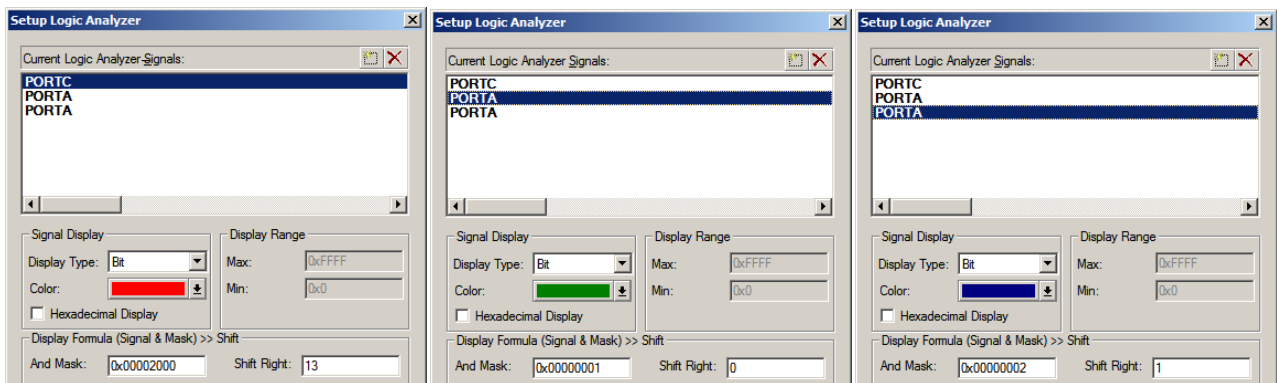
При создании проекта выбираем тип МК STM32F103C8. На шаге настройки работы с библиотеками ARM MDK-Professional, выбираем пункты: CMSIS >> CORE; Device >> Startup; Compiler >> I/O >> STDOUT в выпадающем меню ITM.

На вкладке Debug выбираем Use Simulator. В поле Dialog DLL и Parameter вводим DARMSTM.DLL и -pSTM32F103C8, соответственно. В поле Initialization File вводим имя файла GPIOA_But.ini:

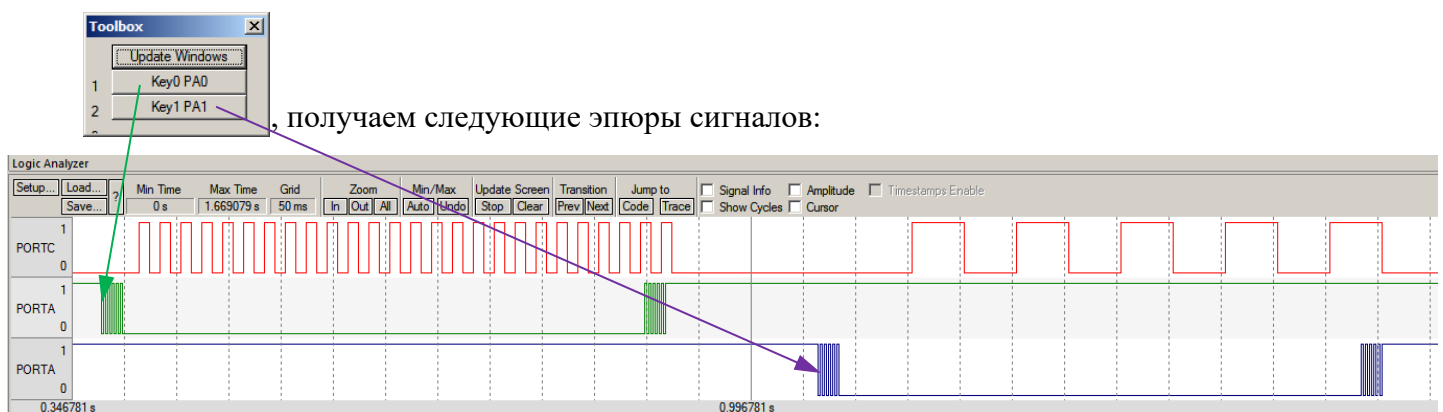


После компиляции и запуска режима отладки используются следующие инструменты: окно Debug (printf) Viewer (View >> Serial Windows >> Debug (printf) Viewer); логический анализатор (View >> Analysis Windows >> Logic Analyzer); окно инструментов (View >> Toolbox Window).

Добавляем сигналы в логический анализатор:



Запускаем выполнение в отладчике ( F5) и при нажатии кнопок в окне инструментов –



получаем следующие эпюры сигналов: где верхняя эпюра это сигнал на выходе PC13 (имитация зажигания светодиода), а нижние это входы PA0 и PA1 – генерация сигналов на эти входы прописана скриптом в файле GPIOA_But.ini.

В окне Debug (printf) Viewer выведены сообщения о нажатии соответствующей кнопки. Поскольку эмуляция на скрипте отладчика описывает дребезг во время перехода из состояния отжата (лог.1) в состояние нажата (лог.0), далее удержание кнопки нажатой полсекунды и дребезг при отпускании, то сообщений о нажатии несколько:

```
Debug (printf) Viewer
clk=72000000 Hz
Key0 PA0 pressed
Key0 PA0 pressed
Key0 PA0 pressed
Key0 PA0 up
1 sec
Key1 PA1 pressed
Key1 PA1 pressed
Key1 PA1 pressed
Key1 PA1 up
2 sec
```

Эмулировать нажатие кнопок с эффектом дребезга в курсовом проекте не требуется, данный пример представлен в целях продемонстрировать возможности симулятора.

6 Литература

- 1 Алфавитно-цифровые индицирующие ЖК-модули на основе контроллера HD44780.
URL: <https://cxem.net/doc/mc/HD44780-rus.pdf>
- 2 Библиотека HD44780 4 строки по 20 символов для STM32. URL: <https://cxem.net/mc/mc398.php>